

```

00001 // Source file: frontend/src/main.ts
00002 import { createApp } from 'vue'
00003 import { createPinia } from 'pinia'
00004 import App from './App.vue'
00005 import './assets/main.css'
00006 import router from './router'
00007 const app = createApp(App)
00008 app.use(createPinia())
00009 app.use(router)
00010 app.mount('#app')
00011 <!-- Source file: frontend/src/App.vue -->
00012 <script setup lang="ts">
00013 import { computed, ref, watch } from 'vue'
00014 import AnalysisWorkspace from '@/components/AnalysisWorkspace.vue'
00015 import GwrfWorkspace from '@/components/GwrfWorkspace.vue'
00016 import SpatialWorkspace from '@/components/SpatialWorkspace.vue'
00017 import { exportCleaning, previewCleaning, previewFile } from '@/services/api'
00018 import type {
00019   CleaningPreviewResponse,
00020   ExportFormat,
00021   ExportTable,
00022   FilePreviewResponse,
00023   MissingAction,
00024   StandardizationMethod,
00025 } from '@/types/analysis'
00026 const selectedFile = ref<File | null>(null)
00027 const activeWorkspace = ref<'cleaning' | 'analysis' | 'spatial' | 'gwrf'>('cleaning')
00028 const result = ref<FilePreviewResponse | null>(null)
00029 const cleaningResult = ref<CleaningPreviewResponse | null>(null)
00030 const errorMessage = ref('')
00031 const isLoading = ref(false)
00032 const isCleaning = ref(false)
00033 const missingAction = ref<MissingAction>('none')
00034 const trimWhitespace = ref(false)
00035 const removeLineBreaks = ref(false)
00036 const exportingTarget = ref('')
00037 const exportMessage = ref('')
00038 const standardizationMethod = ref<StandardizationMethod>('none')
00039 const standardizationColumns = ref<string[]>([])
00040 watch(
00041   [missingAction, trimWhitespace, removeLineBreaks, standardizationMethod, standardizationColumns],
00042   () => {
00043     cleaningResult.value = null
00044     exportMessage.value = ''
00045   },
00046   { deep: true },
00047 )
00048 const hasIssues = computed(() => {
00049   if (!result.value) return false
00050   const quality = result.value.quality
00051   return (
00052     quality.missing_cell_count > 0 ||
00053     quality.duplicate_row_count > 0 ||
00054     quality.columns.some((column) => column.mixed_types)

```

```

00055  )
00056  })
00057  const numericColumns = computed(() =>
00058    (result.value?.quality.columns ?? []))
00059    .filter(
00060      (column) =>
00061        !column.mixed_types &&
00062        column.detected_types.length === 1 &&
00063        column.detected_types[0]?.type === 'number',
00064    )
00065    .map((column) => column.name),
00066  )
00067  const canPreviewCleaning = computed(
00068    () => standardizationMethod.value === 'none' || standardizationColumns.value.length > 0,
00069  )
00070  function setFile(file: File | undefined) {
00071    errorMessage.value = ''
00072    result.value = null
00073    cleaningResult.value = null
00074    exportMessage.value = ''
00075    standardizationMethod.value = 'none'
00076    standardizationColumns.value = []
00077    if (!file) {
00078      selectedFile.value = null
00079      return
00080    }
00081    const extension = file.name.toLowerCase().split('.').pop()
00082    if (!extension || ![ 'csv', 'xlsx' ].includes(extension)) {
00083      selectedFile.value = null
00084      errorMessage.value = '请选择 CSV 或 XLSX 文件。'
00085      return
00086    }
00087    selectedFile.value = file
00088  }
00089  function onFileSelected(event: Event) {
00090    const input = event.target as HTMLInputElement
00091    setFile(input.files?.[0])
00092  }
00093  function onDrop(event: DragEvent) {
00094    setFile(event.dataTransfer?.files[0])
00095  }
00096  async function analyzeSelectedFile() {
00097    if (!selectedFile.value) return
00098    isLoading.value = true
00099    errorMessage.value = ''
00100    try {
00101      result.value = await previewFile(selectedFile.value)
00102      cleaningResult.value = null
00103    } catch (error) {
00104      errorMessage.value = error instanceof Error ? error.message : '文件分析失败。'
00105    } finally {
00106      isLoading.value = false
00107    }
00108  }

```

```

00109 async function previewSelectedCleaning() {
00110   if (!selectedFile.value) return
00111   isCleaning.value = true
00112   errorMessage.value = ''
00113   try {
00114     cleaningResult.value = await previewCleaning(selectedFile.value, {
00115       missingAction: missingAction.value,
00116       trimWhitespace: trimWhitespace.value,
00117       removeLineBreaks: removeLineBreaks.value,
00118       standardizationMethod: standardizationMethod.value,
00119       standardizationColumns: standardizationColumns.value,
00120     })
00121     exportMessage.value = ''
00122   } catch (error) {
00123     errorMessage.value = error instanceof Error ? error.message : '清洗预览失败。'
00124   } finally {
00125     isCleaning.value = false
00126   }
00127 }
00128 async function downloadCleaningResult(outputFormat: ExportFormat, table: ExportTable = 'cleaned') {
00129   if (!selectedFile.value) return
00130   const target = `${outputFormat}-${table}`
00131   exportingTarget.value = target
00132   exportMessage.value = ''
00133   errorMessage.value = ''
00134   try {
00135     const filename = await exportCleaning(
00136       selectedFile.value,
00137       {
00138         missingAction: missingAction.value,
00139         trimWhitespace: trimWhitespace.value,
00140         removeLineBreaks: removeLineBreaks.value,
00141         standardizationMethod: standardizationMethod.value,
00142         standardizationColumns: standardizationColumns.value,
00143       },
00144       outputFormat,
00145       table,
00146     )
00147     exportMessage.value = `已导出 ${filename}`
00148   } catch (error) {
00149     errorMessage.value = error instanceof Error ? error.message : '导出失败。'
00150   } finally {
00151     exportingTarget.value = ''
00152   }
00153 }
00154 function formatFileSize(size: number) {
00155   if (size < 1024) return `${size} B`
00156   if (size < 1024 * 1024) return `${(size / 1024).toFixed(1)} KB`
00157   return `${(size / 1024 / 1024).toFixed(1)} MB`
00158 }
00159 function formatRatio(ratio: number) {
00160   return `${(ratio * 100).toFixed(1)}%`
00161 }
00162 </script>

```

```

00163 <template>
00164 <main class="app-shell">
00165 <nav class="workspace-tabs">
00166 <button
00167   :class="{ active: activeWorkspace === 'cleaning' }"
00168   @click="activeWorkspace = 'cleaning'"
00169 >
00170   数据清洗
00171 </button>
00172 <button
00173   :class="{ active: activeWorkspace === 'analysis' }"
00174   @click="activeWorkspace = 'analysis'"
00175 >
00176   回归分析方法
00177 </button>
00178 <button
00179   :class="{ active: activeWorkspace === 'spatial' }"
00180   @click="activeWorkspace = 'spatial'"
00181 >
00182   空间分析
00183 </button>
00184 <button :class="{ active: activeWorkspace === 'gwrf' }" @click="activeWorkspace = 'gwrf'">
00185   GWRF + SHAP
00186 </button>
00187 </nav>
00188 <div v-show="activeWorkspace === 'cleaning'">
00189 <section class="panel upload-panel compact-upload-panel">
00190 <div class="section-heading">
00191 <div>
00192 <h2>选择数据文件</h2>
00193 </div>
00194 <p>支持 CSV、XLSX，单个文件不超过 20 MB</p>
00195 </div>
00196 <label class="drop-zone compact-drop-zone" @dragover.prevent @drop.prevent="onDrop">
00197 <input type="file" accept=".csv,xlsx" @change="onFileSelected" />
00198 <span class="upload-icon"> ↑ </span>
00199 <strong>点击选择或拖放文件到这里</strong>
00200 </label>
00201 <div v-if="selectedFile" class="selected-file">
00202 <div>
00203 <strong>{{ selectedFile.name }}</strong>
00204 <span>{{ formatFileSize(selectedFile.size) }}</span>
00205 </div>
00206 <button :disabled="isLoading" @click="analyzeSelectedFile">
00207   {{ isLoading ? '分析中...' : '开始分析' }}
00208 </button>
00209 </div>
00210 <p v-if="errorMessage" class="error-message">{{ errorMessage }}</p>
00211 </section>
00212 <template v-if="result">
00213 <section class="summary-grid">
00214 <article class="metric-card">
00215 <span>数据行数</span>
00216 <strong>{{ result.row_count }}</strong>

```

```

00217     </article>
00218     <article class="metric-card">
00219         <span>字段数量</span>
00220         <strong>{{ result.column_count }}</strong>
00221     </article>
00222     <article class="metric-card" :class="{ warning: result.quality.missing_cell_count }">
00223         <span>空值单元格</span>
00224         <strong>{{ result.quality.missing_cell_count }}</strong>
00225     </article>
00226     <article class="metric-card" :class="{ warning: result.quality.duplicate_row_count }">
00227         <span>重复数据行</span>
00228         <strong>{{ result.quality.duplicate_row_count }}</strong>
00229     </article>
00230     <article class="metric-card" :class="{ warning: result.quality.whitespace_cell_count }">
00231         <span>前后空格</span>
00232         <strong>{{ result.quality.whitespace_cell_count }}</strong>
00233     </article>
00234     <article class="metric-card" :class="{ warning: result.quality.line_break_cell_count }">
00235         <span>回车/换行</span>
00236         <strong>{{ result.quality.line_break_cell_count }}</strong>
00237     </article>
00238 </section>
00239 <section class="panel">
00240     <div class="section-heading">
00241         <div>
00242             <p class="step-label">步骤 02</p>
00243             <h2>质量报告</h2>
00244         </div>
00245         <span class="result-badge" :class="{ clean: !hasIssues }">
00246             {{ hasIssues ? '发现需要关注的问题' : '未发现明显问题' }}
00247         </span>
00248     </div>
00249     <div class="quality-table-wrap">
00250         <table class="quality-table">
00251             <thead>
00252                 <tr>
00253                     <th>字段</th>
00254                     <th>pandas 类型</th>
00255                     <th>空值</th>
00256                     <th>检测到的内容类型</th>
00257                     <th>文本问题</th>
00258                     <th>状态</th>
00259                 </tr>
00260             </thead>
00261             <tbody>
00262                 <tr v-for="column in result.quality.columns" :key="column.name">
00263                     <td class="column-name">{{ column.name }}</td>
00264                     <td>
00265                         <code>{{ column.pandas_dtype }}</code>
00266                     </td>
00267                     <td>{{ column.missing_count }} ({{ formatRatio(column.missing_ratio) }}) </td>
00268                     <td>
00269                         <span
00270                             v-for="detected in column.detected_types"

```

```

00271      :key="detected.type"
00272      class="type-chip"
00273      :title="detected.examples.join('、')"
00274      >
00275      {{ detected.type }} · {{ detected.count }}
00276    </span>
00277    <span v-if="!column.detected_types.length" class="muted">无有效值</span>
00278  </td>
00279  <td>
00280    <span v-if="column.whitespace_count" class="issue-chip">
00281      前后空格 {{ column.whitespace_count }}
00282    </span>
00283    <span v-if="column.line_break_count" class="issue-chip">
00284      回车/换行 {{ column.line_break_count }}
00285    </span>
00286    <span v-if="!column.whitespace_count && !column.line_break_count" class="muted">
00287      无
00288    </span>
00289  </td>
00290  <td>
00291    <span v-if="column.mixed_types" class="status warning-text">混合类型</span>
00292    <span v-else class="status ok-text">正常</span>
00293  </td>
00294 </tr>
00295 </tbody>
00296 </table>
00297 </div>
00298 </section>
00299 <section class="panel cleaning-panel">
00300   <div class="section-heading">
00301     <div>
00302       <p class="step-label">步骤 03</p>
00303       <h2>选择清洗规则</h2>
00304     </div>
00305     <p>只生成预览，不会覆盖原文件</p>
00306   </div>
00307   <div class="rule-group">
00308     <h3>空值处理</h3>
00309     <div class="radio-grid">
00310       <label class="rule-option">
00311         <input v-model="missingAction" type="radio" value="none" />
00312         <span><strong>暂不处理</strong><small>保留所有空值</small></span>
00313       </label>
00314       <label class="rule-option">
00315         <input v-model="missingAction" type="radio" value="drop_rows" />
00316         <span><strong>删除空值行</strong><small>删除包含任意空值的整行</small></span>
00317       </label>
00318       <label class="rule-option">
00319         <input v-model="missingAction" type="radio" value="extract_rows" />
00320         <span><strong>摘出为空值表</strong><small>从主表移出并生成独立表</small></span>
00321       </label>
00322       <label class="rule-option">
00323         <input v-model="missingAction" type="radio" value="fill_zero" />
00324         <span><strong>使用 0 替换</strong><small>将所有空值填充为数值 0</small></span>

```

```

00325     </label>
00326   </div>
00327 </div>
00328 <div class="rule-group">
00329   <h3>文本处理</h3>
00330   <div class="checkbox-list">
00331     <label>
00332       <input v-model="trimWhitespace" type="checkbox" />
00333       清除文本前后空格
00334       <small>检测到 {{ result.quality.whitespace_cell_count }} 个单元格</small>
00335     </label>
00336     <label>
00337       <input v-model="removeLineBreaks" type="checkbox" />
00338       清除回车和换行符
00339       <small>检测到 {{ result.quality.line_break_cell_count }} 个单元格</small>
00340     </label>
00341   </div>
00342 </div>
00343 <div class="rule-group">
00344   <h3>数据标准化</h3>
00345   <div class="standardization-options">
00346     <label>
00347       <input v-model="standardizationMethod" type="radio" value="none" />
00348       暂不标准化
00349     </label>
00350     <label>
00351       <input v-model="standardizationMethod" type="radio" value="min_max" />
00352       Min-Max 标准化
00353       <small>将数值缩放至 0-1</small>
00354     </label>
00355     <label>
00356       <input v-model="standardizationMethod" type="radio" value="z_score" />
00357       Z-score 标准化
00358       <small>转换为均值 0、标准差 1</small>
00359     </label>
00360   </div>
00361   <div v-if="standardizationMethod !== 'none'" class="column-selector">
00362     <p>选择需要标准化的数值列: </p>
00363     <label v-for="column in numericColumns" :key="column">
00364       <input v-model="standardizationColumns" type="checkbox" :value="column" />
00365       {{ column }}
00366     </label>
00367     <p v-if="!numericColumns.length" class="validation-message">
00368       当前数据没有可安全标准化的纯数值列。
00369     </p>
00370     <p v-else-if="!standardizationColumns.length" class="validation-message">
00371       请至少选择一个数值列。
00372     </p>
00373   </div>
00374 </div>
00375 <button
00376   class="primary-action"
00377   :disabled="isCleaning || !canPreviewCleaning"
00378   @click="previewSelectedCleaning"

```

```

00379 >
00380 {{ isCleaning ? '正在生成...' : '生成清洗预览' }}
00381 </button>
00382 <div v-if="cleaningResult" class="cleaning-result">
00383   <div class="cleaning-summary">
00384     <span>原始 {{ cleaningResult.original_row_count }} 行</span>
00385     <span>清洗后 {{ cleaningResult.cleaned_row_count }} 行</span>
00386     <span v-if="cleaningResult.extracted_row_count">
00387       摘出 {{ cleaningResult.extracted_row_count }} 行
00388     </span>
00389     <span>修改文本 {{ cleaningResult.summary.text_changed_cell_count }} 格</span>
00390     <span v-if="cleaningResult.summary.standardization_method === 'min_max'">
00391       Min-Max: {{ cleaningResult.summary.standardized_columns.join('、') }}
00392     </span>
00393     <span v-if="cleaningResult.summary.standardization_method === 'z_score'">
00394       Z-score: {{ cleaningResult.summary.standardized_columns.join('、') }}
00395     </span>
00396   </div>
00397   <h3>清洗后主表预览</h3>
00398   <div class="data-table-wrap compact-table">
00399     <table class="data-table">
00400       <thead>
00401         <tr>
00402           <th class="row-number">#</th>
00403           <th v-for="column in cleaningResult.columns" :key="column">{{ column }}</th>
00404         </tr>
00405       </thead>
00406       <tbody>
00407         <tr v-for="(row, rowIndex) in cleaningResult.cleaned_preview" :key="rowIndex">
00408           <td class="row-number">{{ rowIndex + 1 }}</td>
00409           <td v-for="column in cleaningResult.columns" :key="column">
00410             <span v-if="row[column] === null" class="missing-value">空值</span>
00411             <span v-else>{{ row[column] }}</span>
00412           </td>
00413         </tr>
00414       </tbody>
00415     </table>
00416   </div>
00417   <template v-if="cleaningResult.extracted_row_count">
00418     <h3 class="extracted-heading">空值数据表预览</h3>
00419     <p class="muted">
00420       来源数据行: {{ cleaningResult.summary.extracted_row_numbers.join('、') }}
00421     </p>
00422     <div class="data-table-wrap compact-table extracted-table">
00423       <table class="data-table">
00424         <thead>
00425           <tr>
00426             <th class="row-number">#</th>
00427             <th v-for="column in cleaningResult.columns" :key="column">{{ column }}</th>
00428           </tr>
00429         </thead>
00430         <tbody>
00431           <tr v-for="(row, rowIndex) in cleaningResult.extracted_preview" :key="rowIndex">
00432             <td class="row-number">{{ rowIndex + 1 }}</td>

```

```

00433         <td v-for="column in cleaningResult.columns" :key="column">
00434             <span v-if="row[column] === null" class="missing-value">空值</span>
00435             <span v-else>{{ row[column] }}</span>
00436         </td>
00437     </tr>
00438 </tbody>
00439 </table>
00440 </div>
00441 </template>
00442 <div class="export-panel">
00443     <div>
00444         <h3>导出清洗结果</h3>
00445         <p>新文件不会覆盖原文件，文件名自动添加 -dataex。</p>
00446     </div>
00447     <div class="export-actions">
00448         <button
00449             :disabled="!!exportingTarget"
00450             @click="downloadCleaningResult('csv', 'cleaned')"
00451         >
00452             {{ exportingTarget === 'csv-cleaned' ? '导出中...' : '导出主表 CSV' }}
00453         </button>
00454         <button
00455             v-if="cleaningResult.extracted_row_count"
00456             :disabled="!!exportingTarget"
00457             @click="downloadCleaningResult('csv', 'extracted')"
00458         >
00459             {{ exportingTarget === 'csv-extracted' ? '导出中...' : '导出空值表 CSV' }}
00460         </button>
00461         <button
00462             class="xlsx-button"
00463             :disabled="!!exportingTarget"
00464             @click="downloadCleaningResult('xlsx')"
00465         >
00466             {{ exportingTarget === 'xlsx-cleaned' ? '导出中...' : '导出 XLSX' }}
00467         </button>
00468     </div>
00469     <p v-if="exportMessage" class="export-message">{{ exportMessage }}</p>
00470 </div>
00471 </div>
00472 </section>
00473 <section class="panel">
00474     <div class="section-heading">
00475         <div>
00476             <p class="step-label">原始数据</p>
00477             <h2>原始数据预览</h2>
00478         </div>
00479         <p>显示前 {{ result.preview.length }} 行</p>
00480     </div>
00481     <div class="data-table-wrap">
00482         <table class="data-table">
00483             <thead>
00484                 <tr>
00485                     <th class="row-number">#</th>
00486                     <th v-for="column in result.columns" :key="column">{{ column }}</th>

```

```

00487         </tr>
00488     </thead>
00489     <tbody>
00490         <tr v-for="(row, rowIndex) in result.preview" :key="rowIndex">
00491             <td class="row-number">{{ rowIndex + 1 }}</td>
00492             <td v-for="column in result.columns" :key="column">
00493                 <span v-if="row[column] === null" class="missing-value">空值</span>
00494                 <span v-else>{{ row[column] }}</span>
00495             </td>
00496         </tr>
00497     </tbody>
00498 </table>
00499 </div>
00500 </section>
00501 </template>
00502 </div>
00503 <div v-show="activeWorkspace === 'analysis'" class="workspace-view">
00504     <AnalysisWorkspace />
00505 </div>
00506 <div v-show="activeWorkspace === 'spatial'" class="workspace-view">
00507     <SpatialWorkspace />
00508 </div>
00509 <div v-show="activeWorkspace === 'gwr'" class="workspace-view">
00510     <GwrWorkspace />
00511 </div>
00512 </main>
00513 </template>
00514 <!-- Source file: frontend/src/components/AnalysisWorkspace.vue -->
00515 <script setup lang="ts">
00516 import { computed, ref, watch } from 'vue'
00517 import { exportAnalysis, previewFile, runAnalysis } from '@services/api'
00518 import type {
00519     AnalysisMethod,
00520     AnalysisResponse,
00521     ExportFormat,
00522     FilePreviewResponse,
00523 } from '@types/analysis'
00524 const methods: Array<{ value: AnalysisMethod; label: string; description: string }> = [
00525     { value: 'ols', label: 'OLS', description: '连续因变量的普通最小二乘回归' },
00526     { value: 'negative_binomial', label: '负二项回归', description: '非负整数计数型因变量' },
00527     { value: 'pearson', label: 'Pearson', description: '两个数值变量的线性相关' },
00528     { value: 'spearman', label: 'Spearman', description: '两个数值变量的单调相关' },
00529     { value: 'logistic', label: 'Logistic', description: '只有两类结果的二元回归' },
00530 ]
00531 const selectedFile = ref<File | null>(null)
00532 const dataset = ref<FilePreviewResponse | null>(null)
00533 const selectedMethod = ref<AnalysisMethod>('ols')
00534 const dependentColumn = ref('')
00535 const independentColumns = ref<string[]>([])
00536 const correlationColumn = ref('')
00537 const result = ref<AnalysisResponse | null>(null)
00538 const errorMessage = ref('')
00539 const isReading = ref(false)
00540 const isRunning = ref(false)

```

```

00541 const exportingFormat = ref<ExportFormat | ">("")
00542 const exportMessage = ref("")
00543 const numericColumns = computed(() =>
00544   (dataset.value?.quality.columns ?? []).
00545     .filter(
00546       (column) =>
00547         !column.mixed_types &&
00548         column.detected_types.length === 1 &&
00549         column.detected_types[0]?.type === 'number',
00550     )
00551     .map((column) => column.name),
00552 )
00553 const isCorrelation = computed(() => ['pearson', 'spearman'].includes(selectedMethod.value))
00554 const availableIndependentColumns = computed(() =>
00555   numericColumns.value.filter((column) => column !== dependentColumn.value),
00556 )
00557 const canRun = computed(() => {
00558   if (!selectedFile.value || !dependentColumn.value) return false
00559   return isCorrelation.value
00560     ? Boolean(correlationColumn.value)
00561     : independentColumns.value.length > 0
00562 })
00563 watch([selectedMethod, dependentColumn], () => {
00564   result.value = null
00565   errorMessage.value = ""
00566   exportMessage.value = ""
00567   independentColumns.value = independentColumns.value.filter(
00568     (column) => column !== dependentColumn.value,
00569   )
00570   if (correlationColumn.value === dependentColumn.value) correlationColumn.value = ""
00571 })
00572 async function loadSelectedFile(file: File | undefined) {
00573   if (!file) return
00574   selectedFile.value = file
00575   dataset.value = null
00576   result.value = null
00577   dependentColumn.value = ""
00578   independentColumns.value = []
00579   correlationColumn.value = ""
00580   errorMessage.value = ""
00581   isReading.value = true
00582   try {
00583     dataset.value = await previewFile(file)
00584   } catch (error) {
00585     selectedFile.value = null
00586     errorMessage.value = error instanceof Error ? error.message : '读取字段失败。'
00587   } finally {
00588     isReading.value = false
00589   }
00590 }
00591 function onFileSelected(event: Event) {
00592   void loadSelectedFile((event.target as HTMLInputElement).files?.[0])
00593 }
00594 function onDrop(event: DragEvent) {

```

```

00595 void loadSelectedFile(event.dataTransfer?.files[0])
00596 }
00597 async function executeAnalysis() {
00598   if (!selectedFile.value || !canRun.value) return
00599   isRunning.value = true
00600   result.value = null
00601   errorMessage.value = ""
00602   try {
00603     result.value = await runAnalysis(
00604       selectedFile.value,
00605       selectedMethod.value,
00606       dependentColumn.value,
00607       isCorrelation.value ? [correlationColumn.value] : independentColumns.value,
00608     )
00609   } catch (error) {
00610     errorMessage.value = error instanceof Error ? error.message : '分析失败。'
00611   } finally {
00612     isRunning.value = false
00613   }
00614 }
00615 async function downloadAnalysisResult(outputFormat: ExportFormat) {
00616   if (!selectedFile.value || !result.value) return
00617   exportingFormat.value = outputFormat
00618   errorMessage.value = ""
00619   exportMessage.value = ""
00620   try {
00621     const filename = await exportAnalysis(
00622       selectedFile.value,
00623       selectedMethod.value,
00624       dependentColumn.value,
00625       isCorrelation.value ? [correlationColumn.value] : independentColumns.value,
00626       outputFormat,
00627     )
00628     exportMessage.value = `已导出 ${filename}`
00629   } catch (error) {
00630     errorMessage.value = error instanceof Error ? error.message : '结果导出失败。'
00631   } finally {
00632     exportingFormat.value = ""
00633   }
00634 }
00635 function formatNumber(value: number | null) {
00636   if (value === null) return '—'
00637   if (value !== 0 && Math.abs(value) < 0.0001) return value.toExponential(3)
00638   return value.toFixed(6)
00639 }
00640 function correlationStrength(value: number | null) {
00641   if (value === null) return '无法判断'
00642   const magnitude = Math.abs(value)
00643   if (magnitude >= 0.7) return '强相关'
00644   if (magnitude >= 0.4) return '中等相关'
00645   return '弱相关'
00646 }
00647 </script>
00648 <template>

```

```

00649 <section class="panel analysis-intro compact-upload-panel">
00650 <div class="section-heading">
00651 <div>
00652 <h2>回归分析方法</h2>
00653 </div>
00654 <p>OLS、负二项、Logistic 回归，以及 Pearson、Spearman 相关性</p>
00655 </div>
00656 <label class="drop-zone compact-drop-zone" @dragover.prevent @drop.prevent="onDrop">
00657 <input type="file" accept=".csv,.xlsx" @change="onFileSelected" />
00658 <span class="upload-icon"> ↑ </span>
00659 <strong>{{ isReading ? '正在读取字段...' : '点击选择或拖放分析数据' }}</strong>
00660 </label>
00661 <p v-if="errorMessage" class="error-message">{{ errorMessage }}</p>
00662 </section>
00663 <template v-if="dataset">
00664 <section class="panel">
00665 <div class="analysis-file-summary">
00666 <span>{{ dataset.filename }}</span>
00667 <span>{{ dataset.row_count }} 行</span>
00668 <span>{{ numericColumns.length }} 个可用数值列</span>
00669 </div>
00670 <h3 class="analysis-section-title">选择分析方法</h3>
00671 <div class="method-grid">
00672 <label v-for="method in methods" :key="method.value" class="method-card">
00673 <input v-model="selectedMethod" type="radio" :value="method.value" />
00674 <strong>{{ method.label }}</strong>
00675 <small>{{ method.description }}</small>
00676 </label>
00677 </div>
00678 <div class="variable-grid">
00679 <label>
00680 <span>{{ isCorrelation ? '变量 Y' : '因变量 Y' }}</span>
00681 <select v-model="dependentColumn">
00682 <option value="" disabled>请选择数值列</option>
00683 <option v-for="column in numericColumns" :key="column" :value="column">
00684 {{ column }}
00685 </option>
00686 </select>
00687 </label>
00688 <label v-if="isCorrelation">
00689 <span>变量 X</span>
00690 <select v-model="correlationColumn">
00691 <option value="" disabled>请选择另一个数值列</option>
00692 <option v-for="column in availableIndependentColumns" :key="column" :value="column">
00693 {{ column }}
00694 </option>
00695 </select>
00696 </label>
00697 <div v-else class="independent-selector">
00698 <span>自变量 X（可多选）</span>
00699 <div>
00700 <label v-for="column in availableIndependentColumns" :key="column">
00701 <input v-model="independentColumns" type="checkbox" :value="column" />
00702 {{ column }}

```

```

00703     </label>
00704   </div>
00705 </div>
00706 </div>
00707 <p v-if="!numericColumns.length" class="analysis-warning">
00708   当前文件没有可安全分析的纯数值列，请先清洗或转换数据类型。
00709 </p>
00710 <button class="primary-action" :disabled="isRunning || !canRun" @click="executeAnalysis">
00711   {{ isRunning ? '正在计算...' : '运行分析' }}
00712 </button>
00713 </section>
00714 <section v-if="result" class="panel analysis-result">
00715   <div class="section-heading">
00716     <div>
00717       <p class="step-label">RESULT</p>
00718       <h2>分析结果</h2>
00719     </div>
00720     <p>有效样本 {{ result.observations }}, 排除缺失行 {{ result.dropped_rows }}</p>
00721   </div>
00722   <div
00723     v-if="result.diagnostics"
00724     class="diagnostics-panel"
00725     :class="{ invalid: !result.diagnostics.valid_inference }"
00726   >
00727     <div class="diagnostics-heading">
00728       <div>
00729         <strong>
00730           {{ result.diagnostics.valid_inference ? '模型推断有效' : '模型推断不可靠' }}
00731         </strong>
00732         <small>
00733           {{ result.diagnostics.converged ? '模型已收敛' : '模型未收敛' }}
00734         </small>
00735       </div>
00736       <div class="diagnostic-values">
00737         <span>标准化条件数 {{ formatNumber(result.diagnostics.condition_number) }}</span>
00738         <span>原始条件数 {{ formatNumber(result.diagnostics.raw_condition_number) }}</span>
00739         <span>最大 VIF {{ formatNumber(result.diagnostics.max_vif) }}</span>
00740         <span>尺度比 {{ formatNumber(result.diagnostics.scale_ratio) }}</span>
00741       </div>
00742     </div>
00743     <ul v-if="result.diagnostics.warnings.length">
00744       <li v-for="warning in result.diagnostics.warnings" :key="warning">{{ warning }}</li>
00745     </ul>
00746     <p v-else>未检测到收敛、非有限数值或明显共线性问题。</p>
00747     <details v-if="result.diagnostics.vif.length">
00748       <summary>查看各变量 VIF</summary>
00749       <div class="vif-list">
00750         <span v-for="item in result.diagnostics.vif" :key="item.column">
00751           {{ item.column }}: {{ formatNumber(item.vif) }}
00752         </span>
00753       </div>
00754     </details>
00755   </div>
00756   <div v-if="result.correlation" class="correlation-result">

```

```

00757 <article>
00758   <span>相关系数</span>
00759   <strong>{{ formatNumber(result.correlation.coefficient) }}</strong>
00760   <small>{{ correlationStrength(result.correlation.coefficient) }}</small>
00761 </article>
00762 <article>
00763   <span>p 值</span>
00764   <strong>{{ formatNumber(result.correlation.p_value) }}</strong>
00765   <small>{{ (result.correlation.p_value ?? 1) < 0.05 ? '统计显著' : '未达到 0.05' }}</small>
00766 </article>
00767 </div>
00768 <template v-if="result.regression">
00769   <div class="model-metrics">
00770     <article v-if="result.regression.metrics.r_squared !== null">
00771       <span>R2</span><strong>{{ formatNumber(result.regression.metrics.r_squared) }}</strong>
00772     </article>
00773     <article v-if="result.regression.metrics.adjusted_r_squared !== null">
00774       <span>调整 R2</span>
00775       <strong>{{ formatNumber(result.regression.metrics.adjusted_r_squared) }}</strong>
00776     </article>
00777     <article v-if="result.regression.metrics.pseudo_r_squared !== null">
00778       <span>伪 R2</span>
00779       <strong>{{ formatNumber(result.regression.metrics.pseudo_r_squared) }}</strong>
00780     </article>
00781     <article>
00782       <span>AIC</span><strong>{{ formatNumber(result.regression.metrics.aic) }}</strong>
00783     </article>
00784     <article>
00785       <span>BIC</span><strong>{{ formatNumber(result.regression.metrics.bic) }}</strong>
00786     </article>
00787   </div>
00788   <p v-if="result.regression.binary_mapping" class="binary-mapping">
00789     Logistic 编码:
00790     <span v-for="(value, label) in result.regression.binary_mapping" :key="label">
00791       {{ label }} → {{ value }}
00792     </span>
00793   </p>
00794   <div class="quality-table-wrap">
00795     <table class="quality-table">
00796       <thead>
00797         <tr>
00798           <th>变量</th>
00799           <th>系数</th>
00800           <th>标准误</th>
00801           <th>统计量</th>
00802           <th>p 值</th>
00803           <th>95% 置信区间</th>
00804           <th v-if="selectedMethod === 'logistic'">优势比</th>
00805           <th v-if="selectedMethod === 'negative_binomial'">发生率比</th>
00806         </tr>
00807       </thead>
00808       <tbody>
00809         <tr v-for="coefficient in result.regression.coefficients" :key="coefficient.term">
00810           <td class="column-name">{{ coefficient.term }}</td>

```

```

00811      <td>{{ formatNumber(coefficient.estimate) }}</td>
00812      <td>{{ formatNumber(coefficient.standard_error) }}</td>
00813      <td>{{ formatNumber(coefficient.statistic) }}</td>
00814      <td>{{ formatNumber(coefficient.p_value) }}</td>
00815      <td>
00816          {{ formatNumber(coefficient.confidence_low) }} ~
00817          {{ formatNumber(coefficient.confidence_high) }}
00818      </td>
00819      <td v-if="['logistic', 'negative_binomial'].includes(selectedMethod)">
00820          {{ formatNumber(coefficient.effect_ratio) }}
00821      </td>
00822  </tr>
00823 </tbody>
00824 </table>
00825 </div>
00826 </template>
00827 <p class="analysis-disclaimer">
00828     统计显著性和相关性不代表因果关系；正式结论还需检查模型假设、样本设计与业务背景。
00829 </p>
00830 <div class="export-panel analysis-export-panel">
00831     <div>
00832         <h3>导出分析结果</h3>
00833         <p>文件名自动添加 -analysis-dataex; XLSX 包含概览和系数工作表。</p>
00834     </div>
00835     <div class="export-actions">
00836         <button :disabled="!!exportingFormat" @click="downloadAnalysisResult('csv')">
00837             {{ exportingFormat === 'csv' ? '导出中...' : '导出 CSV' }}
00838         </button>
00839         <button
00840             class="xlsx-button"
00841             :disabled="!!exportingFormat"
00842             @click="downloadAnalysisResult('xlsx')"
00843         >
00844             {{ exportingFormat === 'xlsx' ? '导出中...' : '导出 XLSX' }}
00845         </button>
00846     </div>
00847     <p v-if="exportMessage" class="export-message">{{ exportMessage }}</p>
00848 </div>
00849 </section>
00850 </template>
00851 </template>
00852 <style scoped>
00853 .analysis-file-summary,
00854 .model-metrics,
00855 .correlation-result {
00856     display: flex;
00857     flex-wrap: wrap;
00858     gap: 10px;
00859 }
00860 .analysis-file-summary span,
00861 .binary-mapping span {
00862     padding: 7px 10px;
00863     border: 1px solid #c9ccce;
00864     border-radius: 2px;

```

```
00865 color: #34373a;
00866 background: #f1f2f2;
00867 font-size: 0.8rem;
00868 font-weight: 700;
00869 }
00870 .analysis-section-title {
00871   margin: 26px 0 14px;
00872 }
00873 .method-grid {
00874   display: grid;
00875   grid-template-columns: repeat(5, 1fr);
00876   gap: 9px;
00877 }
00878 .method-card {
00879   display: grid;
00880   grid-template-columns: auto 1fr;
00881   align-content: start;
00882   align-items: center;
00883   column-gap: 8px;
00884   min-height: 104px;
00885   padding: 14px;
00886   border: 1px solid #c9ccce;
00887   border-radius: 2px;
00888   color: #34373a;
00889   background: #f7f7f7;
00890   cursor: pointer;
00891 }
00892 .method-card:has(input:checked) {
00893   border-color: #292b2d;
00894   background: #e2e4e5;
00895   box-shadow: inset 3px 0 0 #292b2d;
00896 }
00897 .method-card input {
00898   accent-color: #2d2f31;
00899 }
00900 .method-card strong,
00901 .method-card small {
00902   display: block;
00903 }
00904 .method-card strong {
00905   margin: 0;
00906 }
00907 .method-card small {
00908   grid-column: 1 / -1;
00909   margin-top: 8px;
00910   color: #686c6f;
00911   line-height: 1.35;
00912 }
00913 .variable-grid {
00914   display: grid;
00915   grid-template-columns: repeat(2, 1fr);
00916   gap: 14px;
00917   margin: 22px 0;
00918 }
```

```
00919 .variable-grid > label > span,
00920 .independent-selector > span {
00921   display: block;
00922   margin-bottom: 8px;
00923   font-size: 0.8rem;
00924   font-weight: 800;
00925 }
00926 .variable-grid select {
00927   width: 100%;
00928   padding: 11px;
00929   border: 1px solid #c9ccce;
00930   border-radius: 2px;
00931   color: #34373a;
00932   background: white;
00933 }
00934 .independent-selector > div {
00935   display: flex;
00936   flex-wrap: wrap;
00937   gap: 7px;
00938 }
00939 .independent-selector label {
00940   padding: 9px 11px;
00941   border: 1px solid #c9ccce;
00942   border-radius: 2px;
00943   color: #34373a;
00944   background: #f1f2f2;
00945   cursor: pointer;
00946 }
00947 .analysis-warning {
00948   color: #9a5c12;
00949 }
00950 .correlation-result article,
00951 .model-metrics article {
00952   min-width: 160px;
00953   padding: 18px;
00954   border: 1px solid #c9ccce;
00955   border-radius: 2px;
00956   background: #f1f2f2;
00957 }
00958 .correlation-result span,
00959 .correlation-result strong,
00960 .correlation-result small,
00961 .model-metrics span,
00962 .model-metrics strong {
00963   display: block;
00964 }
00965 .correlation-result strong,
00966 .model-metrics strong {
00967   margin-top: 7px;
00968   font-size: 1.35rem;
00969 }
00970 .correlation-result small {
00971   margin-top: 5px;
00972   color: #686c6f;
```

```
00973 }
00974 .model-metrics {
00975   margin-bottom: 22px;
00976 }
00977 .binary-mapping {
00978   margin-bottom: 18px;
00979 }
00980 .analysis-disclaimer {
00981   margin: 20px 0 0;
00982   padding: 12px;
00983   border-radius: 2px;
00984   color: #6f6756;
00985   background: #f8f4e8;
00986   font-size: 0.78rem;
00987 }
00988 .analysis-export-panel {
00989   margin-top: 18px;
00990 }
00991 .diagnostics-panel {
00992   margin-bottom: 20px;
00993   padding: 16px;
00994   border: 1px solid #c9ccce;
00995   border-radius: 2px;
00996   background: #f1f2f2;
00997 }
00998 .diagnostics-panel.invalid {
00999   border-color: #e1a39b;
01000   background: #fff0ed;
01001 }
01002 .diagnostics-heading {
01003   display: flex;
01004   align-items: flex-start;
01005   justify-content: space-between;
01006   gap: 16px;
01007 }
01008 .diagnostics-heading strong,
01009 .diagnostics-heading small {
01010   display: block;
01011 }
01012 .diagnostics-heading small {
01013   margin-top: 4px;
01014   color: #686c6f;
01015 }
01016 .diagnostic-values,
01017 .vif-list {
01018   display: flex;
01019   flex-wrap: wrap;
01020   gap: 7px;
01021 }
01022 .diagnostic-values span,
01023 .vif-list span {
01024   padding: 6px 9px;
01025   border: 1px solid #c9ccce;
01026   border-radius: 2px;
```

```

01027 background: rgb(255 255 255 / 72%);
01028 font-size: 0.76rem;
01029 font-weight: 700;
01030 }
01031 .diagnostics-panel ul {
01032   margin-bottom: 8px;
01033   padding-left: 20px;
01034 }
01035 .diagnostics-panel li + li {
01036   margin-top: 6px;
01037 }
01038 .diagnostics-panel details {
01039   margin-top: 10px;
01040 }
01041 .diagnostics-panel summary {
01042   cursor: pointer;
01043   font-weight: 700;
01044 }
01045 .vif-list {
01046   margin-top: 10px;
01047 }
01048 @media (max-width: 900px) {
01049   .method-grid {
01050     grid-template-columns: repeat(2, 1fr);
01051   }
01052 }
01053 @media (max-width: 640px) {
01054   .variable-grid {
01055     align-items: flex-start;
01056     grid-template-columns: 1fr;
01057   }
01058 }
01059 </style>
01060 <!-- Source file: frontend/src/components/GwrfWorkspace.vue -->
01061 <script setup lang="ts">
01062 import { computed, nextTick, ref, watch } from 'vue'
01063 import {
01064   exportGwrf,
01065   optimizeGwrfBandwidth,
01066   optimizeGwrfParameters,
01067   previewFile,
01068   runGwrf,
01069 } from '@services/api'
01070 import type {
01071   CoordinateType,
01072   ExportFormat,
01073   FilePreviewResponse,
01074   GwrfBandwidthOptimizationResponse,
01075   GwrfFitMethod,
01076   GwrfOptions,
01077   GwrfParameterOptimizationResponse,
01078   GwrfResponse,
01079 } from '@types/analysis'
01080 const selectedFile = ref<File | null>(null)

```

```

01081 const dataset = ref<FilePreviewResponse | null>(null)
01082 const coordinateType = ref<CoordinateType>('geographic')
01083 const xColumn = ref('')
01084 const yColumn = ref('')
01085 const dependentColumn = ref('')
01086 const independentColumns = ref<string[]>([])
01087 const bandwidth = ref<number | null>(null)
01088 const fitMethod = ref<GwrfFitMethod>('in_sample')
01089 const nEstimators = ref(200)
01090 const maxDepth = ref(10)
01091 const unlimitedDepth = ref(false)
01092 const minSamplesSplit = ref(5)
01093 const parameterMode = ref<'manual' | 'automatic'>('automatic')
01094 const parameterOptimization = ref<GwrfParameterOptimizationResponse | null>(null)
01095 const bandwidthCandidatesText = ref('')
01096 const bandwidthOptimization = ref<GwrfBandwidthOptimizationResponse | null>(null)
01097 const calculateShap = ref(false)
01098 const calculateShapInteractions = ref(false)
01099 const shapInteractionColumns = ref<string[]>([])
01100 const result = ref<GwrfResponse | null>(null)
01101 const errorMessage = ref('')
01102 const exportMessage = ref('')
01103 const isReading = ref(false)
01104 const isOptimizingParameters = ref(false)
01105 const isOptimizingBandwidth = ref(false)
01106 const isRunning = ref(false)
01107 const exportingFormat = ref<ExportFormat | ">('')
01108 const numericColumns = computed(() =>
01109   (dataset.value?.quality.columns ?? []).filter(
01110     (column) =>
01111       !column.mixed_types &&
01112       column.detected_types.length === 1 &&
01113       column.detected_types[0]?.type === 'number',
01114   )
01115   .map((column) => column.name),
01116 )
01117 const availableIndependentColumns = computed(() =>
01118   numericColumns.value.filter(
01119     (column) =>
01120       column !== dependentColumn.value && column !== xColumn.value && column !== yColumn.value,
01121   ),
01122 )
01123 const maximumBandwidth = computed(() => {
01124   const count = dataset.value?.row_count ?? 3
01125   return fitMethod.value === 'loocv' ? Math.max(3, count - 1) : Math.max(3, count)
01126 })
01127 const bandwidthCandidates = computed(() => {
01128   const tokens = bandwidthCandidatesText.value
01129     .trim()
01130     .split(/[, \s]+/)
01131     .filter(Boolean)
01132   const values = tokens.map(Number)
01133   if (values.some((value) => !Number.isInteger(value))) return []

```

```

01135 return [...new Set(values)]
01136 })
01137 const validBandwidthCandidates = computed(
01138   () =>
01139     bandwidthCandidates.value.length > 0 &&
01140     bandwidthCandidates.value.every(
01141       (candidate) =>
01142         candidate >= Math.max(3, minSamplesSplit.value) &&
01143         candidate < (dataset.value?.row_count ?? 1),
01144     ),
01145 )
01146 const parametersReady = computed(
01147   () => parameterMode.value === 'manual' || parameterOptimization.value !== null,
01148 )
01149 const validFinalBandwidth = computed(
01150   () =>
01151     typeof bandwidth.value === 'number' &&
01152     bandwidth.value >= 3 &&
01153     bandwidth.value <= maximumBandwidth.value &&
01154     minSamplesSplit.value <= bandwidth.value,
01155 )
01156 const canOptimizeParameters = computed(() =>
01157   Boolean(selectedFile.value && dependentColumn.value && independentColumns.value.length),
01158 )
01159 const canOptimizeBandwidth = computed(
01160   () =>
01161     Boolean(
01162       selectedFile.value &&
01163       xColumn.value &&
01164       yColumn.value &&
01165       xColumn.value !== yColumn.value &&
01166       dependentColumn.value &&
01167       independentColumns.value.length,
01168     ) &&
01169     parametersReady.value &&
01170     validBandwidthCandidates.value &&
01171     !isOptimizingParameters.value,
01172 )
01173 const canRun = computed(
01174   () =>
01175     Boolean(
01176       selectedFile.value &&
01177       xColumn.value &&
01178       yColumn.value &&
01179       xColumn.value !== yColumn.value &&
01180       dependentColumn.value &&
01181       independentColumns.value.length,
01182     ) &&
01183     parametersReady.value &&
01184     !isOptimizingParameters.value &&
01185     validFinalBandwidth.value &&
01186     nEstimators.value >= 10 &&
01187     minSamplesSplit.value >= 2 &&
01188     (!calculateShapInteractions.value || shapInteractionColumns.value.length >= 2),

```

```

01189 )
01190 const localColumns = computed(() => Object.keys(result.value?.local_preview[0] ?? {}))
01191 const maximumImportance = computed(() =>
01192   Math.max(
01193     ...(result.value?.importance_summary.map((item) => item.relative_importance ?? 0) ?? [0]),
01194   ),
01195 )
01196 function detectColumn(columns: string[], candidates: string[]) {
01197   const normalized = new Set(candidates.map((candidate) => candidate.toLowerCase()))
01198   return columns.find((column) => normalized.has(column.trim().toLowerCase())) ?? ""
01199 }
01200 function detectCoordinates(columns: string[]) {
01201   const longitude = detectColumn(columns, ['longitude', 'lon', 'lng', '经度'])
01202   const latitude = detectColumn(columns, ['latitude', 'lat', '纬度'])
01203   if (longitude && latitude) {
01204     coordinateType.value = 'geographic'
01205     xColumn.value = longitude
01206     yColumn.value = latitude
01207     return
01208   }
01209   const projectedX = detectColumn(columns, ['x', 'x_coord', 'x_coordinate', '投影 x', '横坐标'])
01210   const projectedY = detectColumn(columns, ['y', 'y_coord', 'y_coordinate', '投影 y', '纵坐标'])
01211   if (projectedX && projectedY) {
01212     coordinateType.value = 'projected'
01213     xColumn.value = projectedX
01214     yColumn.value = projectedY
01215   }
01216 }
01217 watch(
01218   [
01219     coordinateType,
01220     xColumn,
01221     yColumn,
01222     dependentColumn,
01223     bandwidth,
01224     fitMethod,
01225     nEstimators,
01226     maxDepth,
01227     unlimitedDepth,
01228     minSamplesSplit,
01229     bandwidthCandidatesText,
01230     calculateShap,
01231     calculateShapInteractions,
01232     shapInteractionColumns,
01233   ],
01234   () => {
01235     result.value = null
01236     exportMessage.value = ""
01237     const validColumns = independentColumns.value.filter((column) =>
01238       availableIndependentColumns.value.includes(column),
01239     )
01240     if (
01241       validColumns.length !== independentColumns.value.length ||
01242       validColumns.some((column, index) => column !== independentColumns.value[index])

```

```

01243   ){
01244     independentColumns.value = validColumns
01245   }
01246   if (typeof bandwidth.value === 'number' && bandwidth.value > maximumBandwidth.value) {
01247     bandwidth.value = maximumBandwidth.value
01248   }
01249 },
01250 )
01251 watch(
01252   independentColumns,
01253   () => {
01254     result.value = null
01255     exportMessage.value = ""
01256     parameterOptimization.value = null
01257     bandwidthOptimization.value = null
01258     shapInteractionColumns.value = shapInteractionColumns.value.filter((column) =>
01259       independentColumns.value.includes(column),
01260     )
01261   },
01262   { deep: true },
01263 )
01264 watch(dependentColumn, () => {
01265   parameterOptimization.value = null
01266   bandwidthOptimization.value = null
01267 })
01268 watch(
01269   [
01270     coordinateType,
01271     xColumn,
01272     yColumn,
01273     bandwidthCandidatesText,
01274     nEstimators,
01275     maxDepth,
01276     unlimitedDepth,
01277     minSamplesSplit,
01278   ],
01279   () => {
01280     bandwidthOptimization.value = null
01281   },
01282 )
01283 watch(calculateShap, (enabled) => {
01284   if (!enabled) {
01285     calculateShapInteractions.value = false
01286     shapInteractionColumns.value = []
01287   }
01288 })
01289 watch(calculateShapInteractions, (enabled) => {
01290   if (!enabled) shapInteractionColumns.value = []
01291 })
01292 async function loadSelectedFile(file: File | undefined) {
01293   if (!file) return
01294   selectedFile.value = file
01295   dataset.value = null
01296   result.value = null

```

```

01297 independentColumns.value = []
01298 shapInteractionColumns.value = []
01299 parameterOptimization.value = null
01300 bandwidthOptimization.value = null
01301 dependentColumn.value = ""
01302 xColumn.value = ""
01303 yColumn.value = ""
01304 bandwidth.value = null
01305 bandwidthCandidatesText.value = ""
01306 errorMessage.value = ""
01307 isReading.value = true
01308 try {
01309     dataset.value = await previewFile(file)
01310     detectCoordinates(numericColumns.value)
01311 } catch (error) {
01312     errorMessage.value = error instanceof Error ? error.message : '读取字段失败。'
01313 } finally {
01314     isReading.value = false
01315 }
01316 }
01317 function onFileSelected(event: Event) {
01318     void loadSelectedFile((event.target as HTMLInputElement).files?.[0])
01319 }
01320 function onDrop(event: DragEvent) {
01321     void loadSelectedFile(event.dataTransfer?.files[0])
01322 }
01323 function currentOptions(): GwrfOptions {
01324     return {
01325         coordinateType: coordinateType.value,
01326         xColumn: xColumn.value,
01327         yColumn: yColumn.value,
01328         dependentColumn: dependentColumn.value,
01329         independentColumns: independentColumns.value,
01330         bandwidth:
01331             typeof bandwidth.value === 'number' ? bandwidth.value : Math.max(3, minSamplesSplit.value),
01332         fitMethod: fitMethod.value,
01333         nEstimators: nEstimators.value,
01334         maxDepth: unlimitedDepth.value ? null : maxDepth.value,
01335         minSamplesSplit: minSamplesSplit.value,
01336         optimizeParameters: false,
01337         optimizeBandwidth: false,
01338         bandwidthCandidates: bandwidthCandidates.value,
01339         calculateShap: calculateShap.value,
01340         calculateShapInteractions: calculateShapInteractions.value,
01341         shapInteractionColumns: shapInteractionColumns.value,
01342     }
01343 }
01344 async function executeBandwidthOptimization() {
01345     if (!selectedFile.value || !canOptimizeBandwidth.value) return
01346     isOptimizingBandwidth.value = true
01347     bandwidthOptimization.value = null
01348     result.value = null
01349     errorMessage.value = ""
01350     try {

```

```

01351     const optimized = await optimizeGwrfBandwidth(selectedFile.value, currentOptions())
01352     bandwidth.value = optimized.best_bandwidth
01353     await nextTick()
01354     bandwidthOptimization.value = optimized
01355   } catch (error) {
01356     errorMessage.value = error instanceof Error ? error.message : '带宽寻优失败。'
01357   } finally {
01358     isOptimizingBandwidth.value = false
01359   }
01360 }
01361 async function executeParameterOptimization() {
01362   if (isOptimizingParameters.value || !selectedFile.value || !canOptimizeParameters.value) return
01363   isOptimizingParameters.value = true
01364   parameterOptimization.value = null
01365   result.value = null
01366   errorMessage.value = ''
01367   try {
01368     const optimized = await optimizeGwrfParameters(
01369       selectedFile.value,
01370       dependentColumn.value,
01371       independentColumns.value,
01372     )
01373     parameterOptimization.value = optimized
01374     nEstimators.value = optimized.best_parameters.n_estimators
01375     minSamplesSplit.value = optimized.best_parameters.min_samples_split
01376     unlimitedDepth.value = optimized.best_parameters.max_depth === null
01377     if (optimized.best_parameters.max_depth !== null) {
01378       maxDepth.value = optimized.best_parameters.max_depth
01379     }
01380   } catch (error) {
01381     errorMessage.value = error instanceof Error ? error.message : '随机森林参数寻优失败。'
01382   } finally {
01383     isOptimizingParameters.value = false
01384   }
01385 }
01386 async function executeGwrf() {
01387   if (!selectedFile.value || !canRun.value) return
01388   isRunning.value = true
01389   errorMessage.value = ''
01390   result.value = null
01391   try {
01392     result.value = await runGwrf(selectedFile.value, currentOptions())
01393   } catch (error) {
01394     errorMessage.value = error instanceof Error ? error.message : 'GWRF 拟合失败。'
01395   } finally {
01396     isRunning.value = false
01397   }
01398 }
01399 async function downloadResult(format: ExportFormat) {
01400   if (exportingFormat.value || !selectedFile.value || !result.value) return
01401   exportingFormat.value = format
01402   errorMessage.value = ''
01403   exportMessage.value = ''
01404   try {

```

```

01405     const filename = await exportGwrf(result.value.export_id, format)
01406     exportMessage.value = `已导出 ${filename}`
01407   } catch (error) {
01408     errorMessage.value = error instanceof Error ? error.message : 'GWRF 结果导出失败。'
01409   } finally {
01410     exportingFormat.value = ""
01411   }
01412 }
01413 function formatNumber(value: number | null | undefined) {
01414   if (value === null || value === undefined) return '—'
01415   if (value !== 0 && Math.abs(value) < 0.0001) return value.toExponential(3)
01416   return value.toFixed(6)
01417 }
01418 function importanceWidth(value: number | null) {
01419   if (!value || maximumImportance.value <= 0) return '0%'
01420   return `${(value / maximumImportance.value) * 100}%`
01421 }
01422 </script>
01423 <template>
01424   <section class="panel gwrf-intro compact-upload-panel">
01425     <div class="section-heading">
01426       <div>
01427         <h2>地理加权随机森林</h2>
01428       </div>
01429       <p>局部随机森林、双平方核权重、SHAP 解释与残差空间检验</p>
01430     </div>
01431     <label class="drop-zone compact-drop-zone" @dragover.prevent @drop.prevent="onDrop">
01432       <input type="file" accept=".csv,.xlsx" @change="onFileSelected" />
01433       <span class="upload-icon"> ↑ </span>
01434       <strong>{{ isReading ? '正在读取字段...' : '点击选择或拖放 GWRF 数据' }}</strong>
01435     </label>
01436     <p v-if="errorMessage" class="error-message">{{ errorMessage }}</p>
01437   </section>
01438   <template v-if="dataset">
01439     <section class="panel gwrf-settings">
01440       <div class="gwrf-tags">
01441         <span>{{ dataset.filename }}</span>
01442         <span>{{ dataset.row_count }} 行</span>
01443         <span>{{ numericColumns.length }} 个可用数值列</span>
01444       </div>
01445       <h3>1. 设置坐标和分析变量</h3>
01446       <div class="coordinate-type-picker">
01447         <label><input v-model="coordinateType" type="radio" value="geographic" /> 经纬度</label>
01448         <label><input v-model="coordinateType" type="radio" value="projected" /> 投影坐标</label>
01449       </div>
01450       <div class="gwrf-variable-grid">
01451         <label>
01452           <span>{{ coordinateType === 'geographic' ? '经度字段' : 'X 坐标字段' }}</span>
01453           <select v-model="xColumn">
01454             <option value="">请选择</option>
01455             <option v-for="column in numericColumns" :key="column" :value="column">
01456               {{ column }}
01457             </option>
01458           </select>

```

```

01459 </label>
01460 <label>
01461   <span>{{ coordinateType === 'geographic' ? '纬度字段' : 'Y 坐标字段' }}</span>
01462   <select v-model="yColumn">
01463     <option value="">请选择</option>
01464     <option v-for="column in numericColumns" :key="column" :value="column">
01465       {{ column }}
01466     </option>
01467   </select>
01468 </label>
01469 <label>
01470   <span>因变量</span>
01471   <select v-model="dependentColumn">
01472     <option value="">请选择</option>
01473     <option v-for="column in numericColumns" :key="column" :value="column">
01474       {{ column }}
01475     </option>
01476   </select>
01477 </label>
01478 </div>
01479 <div class="gwrf-independent-selector">
01480   <span>自变量（框选一个或多个）</span>
01481   <div>
01482     <label v-for="column in availableIndependentColumns" :key="column">
01483       <input v-model="independentColumns" type="checkbox" :value="column" /> {{ column }}
01484     </label>
01485   </div>
01486 </div>
01487 <h3>2. 确定随机森林参数</h3>
01488 <div class="gwrf-method-grid parameter-mode-grid">
01489   <label :class="{ selected: parameterMode === 'manual' }">
01490     <input v-model="parameterMode" type="radio" value="manual" />
01491     <strong>直接填写模型参数</strong>
01492     <small>直接填写决策树数量、最大树深和最小分裂样本数。</small>
01493   </label>
01494   <label :class="{ selected: parameterMode === 'automatic' }">
01495     <input v-model="parameterMode" type="radio" value="automatic" />
01496     <strong>自动参数寻优</strong>
01497     <small>先运行三折 GridSearchCV，确认最优参数后再进入最终模型。</small>
01498   </label>
01499 </div>
01500 <div class="gwrf-parameter-grid rf-parameter-grid">
01501   <label
01502     ><span>决策树数量</span>
01503     ><input
01504       v-model.number="nEstimators"
01505       type="number"
01506       min="10"
01507       max="2000"
01508       step="10"
01509       :disabled="parameterMode === 'automatic' && !parameterOptimization"
01510     /></label>
01511   <label
01512     ><span>最大树深</span>

```

```

01513     ><input
01514         v-model.number="maxDepth"
01515         type="number"
01516         min="1"
01517         max="200"
01518         :disabled="unlimitedDepth || (parameterMode === 'automatic' && !parameterOptimization)"
01519     /></label>
01520 <label
01521     ><span>最小分裂样本数</span>
01522     ><input
01523         v-model.number="minSamplesSplit"
01524         type="number"
01525         min="2"
01526         :disabled="parameterMode === 'automatic' && !parameterOptimization"
01527     /></label>
01528 </div>
01529 <div class="gwr-f-switches">
01530     <label>
01531         <input
01532             v-model="unlimitedDepth"
01533             type="checkbox"
01534             :disabled="parameterMode === 'automatic' && !parameterOptimization"
01535         />
01536         最大树深不限制
01537     </label>
01538 </div>
01539 <template v-if="parameterMode === 'automatic'">
01540     <button
01541         class="parameter-action"
01542         :disabled="!canOptimizeParameters || isOptimizingParameters"
01543         @click="executeParameterOptimization"
01544     >
01545         {{ isOptimizingParameters ? '正在执行 12 组参数的三折交叉验证...' : '开始参数寻优' }}
01546     </button>
01547     <div v-if="parameterOptimization" class="parameter-result">
01548         <strong>参数寻优完成，可以继续运行最终模型</strong>
01549         <span>决策树 {{ parameterOptimization.best_parameters.n_estimators }}</span>
01550         <span>最大树深 {{ parameterOptimization.best_parameters.max_depth ?? '不限制' }}</span>
01551         <span>最小分裂样本 {{ parameterOptimization.best_parameters.min_samples_split }}</span>
01552         <details>
01553             <summary>查看全部参数组合</summary>
01554             <div class="table-scroll">
01555                 <table>
01556                     <thead>
01557                         <tr>
01558                             <th>排名</th>
01559                             <th>决策树</th>
01560                             <th>最大深度</th>
01561                             <th>最小分裂</th>
01562                             <th>CV RMSE</th>
01563                         </tr>
01564                     </thead>
01565                     <tbody>
01566                         <tr>

```

```

01567         v-for="item in parameterOptimization.search_results"
01568         :key="'${item.n_estimators}-${item.max_depth}-${item.min_samples_split}'"
01569         >
01570         <td>{{ item.rank }}</td>
01571         <td>{{ item.n_estimators }}</td>
01572         <td>{{ item.max_depth ?? '不限制' }}</td>
01573         <td>{{ item.min_samples_split }}</td>
01574         <td>{{ formatNumber(item.cv_rmse) }}</td>
01575     </tr>
01576 </tbody>
01577 </table>
01578 </div>
01579 </details>
01580 </div>
01581 </template>
01582 <h3>3. 选择最终拟合与评估方法</h3>
01583 <div class="gwr-f-method-grid">
01584     <label :class="{ selected: fitMethod === 'in_sample' }">
01585         <input v-model="fitMethod" type="radio" value="in_sample" />
01586         <strong>样本内拟合</strong>
01587         <small>目标点参与其局部模型训练， $R^2$  表示样本内伪  $R^2$ 。</small>
01588     </label>
01589     <label :class="{ selected: fitMethod === 'loocv' }">
01590         <input v-model="fitMethod" type="radio" value="loocv" />
01591         <strong>LOOCV</strong>
01592         <small>每个局部模型排除目标点， $R^2$  更接近样本外预测能力。</small>
01593     </label>
01594 </div>
01595 <h3>4. 寻找最优带宽</h3>
01596 <div class="gwr-f-parameter-grid">
01597     <label class="bandwidth-candidates">
01598         ><span>候选带宽 K（逗号分隔）</span> </div>
01599         ><input
01600             v-model="bandwidthCandidatesText"
01601             type="text"
01602             placeholder="建议根据单元数量等差的设置带宽"
01603         /></label>
01604 </div>
01605 <button
01606     class="parameter-action"
01607     :disabled="!canOptimizeBandwidth || isOptimizingBandwidth"
01608     @click="executeBandwidthOptimization"
01609 >
01610     {{ isOptimizingBandwidth ? '正在逐个比较候选带宽…' : '开始带宽寻优' }}
01611 </button>
01612 <div v-if="bandwidthOptimization" class="parameter-result bandwidth-optimization-result">
01613     <strong>带宽寻优完成，最优带宽已回填到下一步</strong>
01614     <span>最优 K: {{ bandwidthOptimization.best_bandwidth }}</span>
01615     <span>完整观测: {{ bandwidthOptimization.observations }}</span>
05061     "condition_number": None,
05062     "raw_condition_number": None,
05063     "scale_ratio": None,
05064     "max_vif": None,
05065     "vif": [],

```

```

05066     "warnings": [],
05067 }
05068 else:
05069     diagnostics = build_design_diagnostics(prepared, independent_columns)
05070     diagnostic_warnings = diagnostics["warnings"]
05071     if coordinate_duplicates:
05072         diagnostic_warnings.append(
05073             f"有 {coordinate_duplicates} 行使用重复坐标; KNN/GWR 结果可能受重合点影响。"
05074         )
05075     if weights.n_components > 1:
05076         diagnostic_warnings.append(
05077             f"空间权重包含 {weights.n_components} 个互不连通的分量, 建议检查坐标或增大 K 值。"
05078         )
05079     if method == "moran":
05080         specific = _run_moran(prepared, dependent_column, weights, diagnostics)
05081     elif method == "gwr":
05082         specific = _run_gwr(
05083             prepared,
05084             dependent_column,
05085             independent_columns,
05086             x_column,
05087             y_column,
05088             coordinate_type,
05089             diagnostics,
05090             None if include_full_local_results else 100,
05091             weights,
05092         )
05093     else:
05094         specific = _run_spatial_regression(
05095             prepared,
05096             method,
05097             dependent_column,
05098             independent_columns,
05099             weights,
05100             diagnostics,
05101         )
05102     if method != "moran":
05103         specific["model_selection"] = _model_selection_diagnostics(
05104             prepared,
05105             dependent_column,
05106             independent_columns,
05107             weights,
05108         )
05109     return {
05110         "method": method,
05111         "observations": len(prepared),
05112         "dropped_rows": dropped_rows,
05113         "coordinate_type": coordinate_type,
05114         "x_column": x_column,
05115         "y_column": y_column,
05116         "dependent_column": dependent_column,
05117         "independent_columns": independent_columns,
05118         "weights": {
05119             "type": "knn",

```

```

05120     "neighbors": neighbors,
05121     "transformation": "row_standardized",
05122     "components": int(weights.n_components),
05123 },
05124     **specific,
05125 }
05126 # Source file: backend/src/backend/gwrf.py
05127 import math
05128 import time
05129 from typing import Literal
05130 import numpy as np
05131 import pandas as pd
05132 from esda.moran import Moran
05133 from libpysal.weights import KNN
05134 from sklearn.ensemble import RandomForestRegressor
05135 from sklearn.metrics import mean_squared_error, r2_score
05136 from sklearn.model_selection import GridSearchCV
05137 from sklearn.neighbors import NearestNeighbors
05138 from backend.analysis import safe_float
05139 from backend.quality import is_missing
05140 GwrfFitMethod = Literal["in_sample", "loocv"]
05141 CoordinateType = Literal["geographic", "projected"]
05142 RANDOM_SEED = 42
05143 MORAN_RANDOM_SEED = 20260827
05144 class GwrfError(ValueError):
05145     pass
05146 def _prepare_numeric(
05147     dataframe: pd.DataFrame, columns: list[str]
05148 ) -> tuple[pd.DataFrame, int]:
05149     if len(columns) != len(set(columns)):
05150         raise GwrfError("坐标、因变量和自变量不能重复选择。")
05151     converted: dict[str, pd.Series] = {}
05152     for column in columns:
05153         if column not in dataframe.columns:
05154             raise GwrfError(f"字段"{column}"不存在。")
05155         original = dataframe[column]
05156         numeric = pd.to_numeric(original, errors="coerce")
05157         invalid = ~original.map(is_missing) & numeric.isna()
05158         if invalid.any():
05159             raise GwrfError(f"字段"{column}"包含非数值内容。")
05160         converted[column] = numeric
05161     prepared = pd.DataFrame(converted, index=dataframe.index).dropna()
05162     prepared.insert(0, "__source_row__", prepared.index.to_numpy() + 2)
05163     dropped = int(len(dataframe) - len(prepared))
05164     return prepared.reset_index(drop=True), dropped
05165 def _validate_coordinates(
05166     coordinates: np.ndarray, coordinate_type: CoordinateType
05167 ) -> None:
05168     if not np.isfinite(coordinates).all():
05169         raise GwrfError("坐标中包含无穷值。")
05170     if coordinate_type == "geographic":
05171         if ((coordinates[:, 0] < -180) | (coordinates[:, 0] > 180)).any():
05172             raise GwrfError("经度必须位于 -180 到 180 之间。")
05173         if ((coordinates[:, 1] < -90) | (coordinates[:, 1] > 90)).any():

```

```

05174         raise GwrfError("纬度必须位于 -90 到 90 之间。")
05175 def _neighbor_coordinates(
05176     coordinates: np.ndarray, coordinate_type: CoordinateType
05177 ) -> np.ndarray:
05178     if coordinate_type == "projected":
05179         return coordinates
05180     longitude = np.radians(coordinates[:, 0])
05181     latitude = np.radians(coordinates[:, 1])
05182     return np.column_stack(
05183         (
05184             np.cos(latitude) * np.cos(longitude),
05185             np.cos(latitude) * np.sin(longitude),
05186             np.sin(latitude),
05187         )
05188     )
05189 def _bisquare(distances: np.ndarray) -> np.ndarray:
05190     distance_limit = float(np.max(distances))
05191     if distance_limit <= 0:
05192         return np.ones_like(distances, dtype=float)
05193     weights = np.square(1 - np.square(distances / distance_limit))
05194     weights[distances > distance_limit] = 0
05195     return weights
05196 def _grid_search_parameters(
05197     x: pd.DataFrame,
05198     y: pd.Series,
05199     *,
05200     n_estimators: int,
05201     max_depth: int | None,
05202     min_samples_split: int,
05203 ) -> tuple[dict[str, int | None], list[dict[str, object]]]:
05204     if len(x) < 6:
05205         raise GwrfError("自动调参至少需要 6 条完整观测。")
05206     estimator_values = sorted({max(10, n_estimators // 2), n_estimators})
05207     depth_values = list(
05208         dict.fromkeys([max_depth, max_depth * 2 if max_depth is not None else 20, None])
05209     )
05210     split_values = sorted({2, min_samples_split})
05211     grid = GridSearchCV(
05212         RandomForestRegressor(
05213             random_state=RANDOM_SEED,
05214             bootstrap=True,
05215             n_jobs=1,
05216         ),
05217         {
05218             "n_estimators": estimator_values,
05219             "max_depth": depth_values,
05220             "min_samples_split": split_values,
05221         },
05222         cv=3,
05223         scoring="neg_mean_squared_error",
05224         # A process-based joblib pool relaunches the frozen DataEX-G executable on
05225         # Windows and collides with the desktop single-instance mutex. Keep the
05226         # small parameter grid inside the existing local API process.
05227         n_jobs=1,

```

```

05228 )
05229 grid.fit(x, y)
05230 best_parameters = {
05231     "n_estimators": int(grid.best_params_["n_estimators"]),
05232     "max_depth": (
05233         int(grid.best_params_["max_depth"])
05234         if grid.best_params_["max_depth"] is not None
05235         else None
05236     ),
05237     "min_samples_split": int(grid.best_params_["min_samples_split"]),
05238 }
05239 search_results = []
05240 for index, parameters in enumerate(grid.cv_results_["params"]):
05241     mean_mse = -float(grid.cv_results_["mean_test_score"][index])
05242     search_results.append(
05243         {
05244             "n_estimators": int(parameters["n_estimators"]),
05245             "max_depth": (
05246                 int(parameters["max_depth"])
05247                 if parameters["max_depth"] is not None
05248                 else None
05249             ),
05250             "min_samples_split": int(parameters["min_samples_split"]),
05251             "cv_rmse": safe_float(math.sqrt(max(mean_mse, 0))),
05252             "rank": int(grid.cv_results_["rank_test_score"][index]),
05253         }
05254     )
05255 search_results.sort(key=lambda item: int(item["rank"]))
05256 return best_parameters, search_results
05257 def optimize_gwrf_parameters(
05258     dataframe: pd.DataFrame,
05259     *,
05260     dependent_column: str,
05261     independent_columns: list[str],
05262 ) -> dict[str, object]:
05263     if not independent_columns:
05264         raise GwrfError("请至少选择一个自变量。")
05265     prepared, dropped_rows = _prepare_numeric(
05266         dataframe, [dependent_column, *independent_columns]
05267     )
05268     if len(prepared) < 6:
05269         raise GwrfError("随机森林参数寻优至少需要 6 条变量完整的观测。")
05270     x = prepared[independent_columns]
05271     y = prepared[dependent_column]
05272     if (
05273         not np.isfinite(x.to_numpy(dtype=float)).all()
05274         or not np.isfinite(y.to_numpy(dtype=float)).all()
05275     ):
05276         raise GwrfError("所选变量中包含无穷值。")
05277     if y.nunique() < 2:
05278         raise GwrfError("因变量为常量，无法进行参数寻优。")
05279     for column in independent_columns:
05280         if x[column].nunique() < 2:
05281             raise GwrfError(f"自变量'{column}'为常量。")

```

```

05282 best_parameters, search_results = _grid_search_parameters(
05283     x,
05284     y,
05285     n_estimators=200,
05286     max_depth=10,
05287     min_samples_split=5,
05288 )
05289 return {
05290     "observations": len(prepared),
05291     "dropped_rows": dropped_rows,
05292     "dependent_column": dependent_column,
05293     "independent_columns": independent_columns,
05294     "best_parameters": best_parameters,
05295     "search_results": search_results,
05296     "cv_folds": 3,
05297     "scoring": "negative_mean_squared_error",
05298 }
05299 def _residual_moran(
05300     residuals: np.ndarray, coordinates: np.ndarray
05301 ) -> dict[str, object]:
05302     neighbors = min(5, len(coordinates) - 1)
05303     weights = KNN.from_array(coordinates, k=neighbors)
05304     weights.transform = "r"
05305     random_state = np.random.get_state()
05306     try:
05307         np.random.seed(MORAN_RANDOM_SEED)
05308         moran = Moran(residuals, weights, permutations=999, two_tailed=True)
05309     finally:
05310         np.random.set_state(random_state)
05311     return {
05312         "i": safe_float(moran.I),
05313         "expected_i": safe_float(moran.EI),
05314         "z_score": safe_float(moran.z_sim),
05315         "p_normal": safe_float(moran.p_norm),
05316         "p_permutation": safe_float(moran.p_sim),
05317         "permutations": 999,
05318         "random_seed": MORAN_RANDOM_SEED,
05319     }
05320 def _select_bandwidth(
05321     x: pd.DataFrame,
05322     y: pd.Series,
05323     distance_coordinates: np.ndarray,
05324     *,
05325     candidates: list[int],
05326     parameters: dict[str, int | None],
05327 ) -> tuple[int, list[dict[str, object]]]:
05328     candidates = list(dict.fromkeys(candidates))
05329     if not candidates:
05330         raise GwrfError("请至少输入一个候选带宽。")
05331     minimum = max(3, int(parameters["min_samples_split"]))
05332     invalid = [
05333         candidate for candidate in candidates if not minimum <= candidate < len(x)
05334     ]
05335     if invalid:

```

```

05336     raise GwrfError(
05337         f'"LOOCV 候选带宽必须位于 {minimum} 到 {len(x) - 1} 之间; "'
05338         f'"无效值: {', '.join(map(str, invalid))}。"'
05339     )
05340     maximum = max(candidates)
05341     distances, indices = (
05342         NearestNeighbors(n_neighbors=maximum + 1)
05343         .fit(distance_coordinates)
05344         .kneighbors(distance_coordinates)
05345     )
05346     search_results: list[dict[str, object]] = []
05347     for candidate in candidates:
05348         started_at = time.monotonic()
05349         predictions: list[float] = []
05350         for row_index in range(len(x)):
05351             keep = indices[row_index] != row_index
05352             local_indices = indices[row_index][keep][:candidate]
05353             local_distances = distances[row_index][keep][:candidate]
05354             weights = _bisquare(local_distances)
05355             if np.count_nonzero(weights) < 2:
05356                 weights = np.ones_like(weights)
05357             model = RandomForestRegressor(
05358                 n_estimators=int(parameters["n_estimators"]),
05359                 max_depth=parameters["max_depth"],
05360                 min_samples_split=int(parameters["min_samples_split"]),
05361                 random_state=RANDOM_SEED,
05362                 bootstrap=True,
05363                 oob_score=False,
05364                 n_jobs=1,
05365             )
05366             model.fit(
05367                 x.iloc[local_indices],
05368                 y.iloc[local_indices],
05369                 sample_weight=weights,
05370             )
05371             predictions.append(float(model.predict(x.iloc[[row_index]])[0]))
05372         rmse = math.sqrt(mean_squared_error(y, predictions))
05373         search_results.append(
05374             {
05375                 "bandwidth": candidate,
05376                 "loocv_rmse": safe_float(rmse),
05377                 "elapsed_seconds": round(time.monotonic() - started_at, 3),
05378             }
05379         )
05380     best = min(search_results, key=lambda item: float(item["loocv_rmse"]))
05381     return int(best["bandwidth"]), search_results
05382 def optimize_gwrf_bandwidth(
05383     dataframe: pd.DataFrame,
05384     *,
05385     coordinate_type: CoordinateType,
05386     x_column: str,
05387     y_column: str,
05388     dependent_column: str,
05389     independent_columns: list[str],

```

```

05390     bandwidth_candidates: list[int],
05391     n_estimators: int,
05392     max_depth: int | None,
05393     min_samples_split: int,
05394 ) -> dict[str, object]:
05395     if not independent_columns:
05396         raise GwrfError("请至少选择一个自变量。")
05397     selected = [x_column, y_column, dependent_column, *independent_columns]
05398     prepared, dropped_rows = _prepare_numeric(dataframe, selected)
05399     if len(prepared) < 6:
05400         raise GwrfError("GWRF 带宽寻优至少需要 6 条变量完整的观测。")
05401     if not 10 <= n_estimators <= 2000:
05402         raise GwrfError("决策树数量必须位于 10 到 2000 之间。")
05403     if max_depth is not None and not 1 <= max_depth <= 200:
05404         raise GwrfError("最大树深必须位于 1 到 200 之间，或留空表示不限制。")
05405     if min_samples_split < 2:
05406         raise GwrfError("最小分裂样本数必须大于或等于 2。")
05407     coordinates = prepared[[x_column, y_column]].to_numpy(dtype=float)
05408     _validate_coordinates(coordinates, coordinate_type)
05409     x = prepared[independent_columns]
05410     y = prepared[dependent_column]
05411     if (
05412         not np.isfinite(x.to_numpy(dtype=float)).all()
05413         or not np.isfinite(y.to_numpy(dtype=float)).all()
05414     ):
05415         raise GwrfError("所选变量中包含无穷值。")
05416     if y.nunique() < 2:
05417         raise GwrfError("因变量为常量，无法进行带宽寻优。")
05418     for column in independent_columns:
05419         if x[column].nunique() < 2:
05420             raise GwrfError(f"自变量“{column}”为常量。")
05421     parameters: dict[str, int | None] = {
05422         "n_estimators": n_estimators,
05423         "max_depth": max_depth,
05424         "min_samples_split": min_samples_split,
05425     }
05426     best_bandwidth, search_results = _select_bandwidth(
05427         x,
05428         y,
05429         _neighbor_coordinates(coordinates, coordinate_type),
05430         candidates=bandwidth_candidates,
05431         parameters=parameters,
05432     )
05433     return {
05434         "observations": len(prepared),
05435         "dropped_rows": dropped_rows,
05436         "best_bandwidth": best_bandwidth,
05437         "search_results": search_results,
05438         "rf_parameters": parameters,
05439     }
05440 def run_gwrf(
05441     dataframe: pd.DataFrame,
05442     *,
05443     coordinate_type: CoordinateType,

```

```

05444 x_column: str,
05445 y_column: str,
05446 dependent_column: str,
05447 independent_columns: list[str],
05448 bandwidth: int,
05449 fit_method: GwrfFitMethod,
05450 n_estimators: int = 200,
05451 max_depth: int | None = 10,
05452 min_samples_split: int = 5,
05453 optimize_parameters: bool = False,
05454 optimize_bandwidth: bool = False,
05455 bandwidth_candidates: list[int] | None = None,
05456 calculate_shap: bool = False,
05457 calculate_shap_interactions: bool = False,
05458 shap_interaction_columns: list[str] | None = None,
05459 include_full_local_results: bool = False,
05460 ) -> dict[str, object]:
05461     if not independent_columns:
05462         raise GwrfError("请至少选择一个自变量。")
05463     selected = [x_column, y_column, dependent_column, *independent_columns]
05464     prepared, dropped_rows = _prepare_numeric(dataframe, selected)
05465     if len(prepared) < 6:
05466         raise GwrfError("GWRF 至少需要 6 条变量完整的观测。")
05467     if not optimize_bandwidth:
05468         if not 3 <= bandwidth <= len(prepared):
05469             raise GwrfError(f"带宽必须位于 3 到 {len(prepared)} 之间。")
05470         if fit_method == "loocv" and bandwidth >= len(prepared):
05471             raise GwrfError("LOOCV 带宽必须小于完整观测数。")
05472     if not 10 <= n_estimators <= 2000:
05473         raise GwrfError("决策树数量必须位于 10 到 2000 之间。")
05474     if max_depth is not None and not 1 <= max_depth <= 200:
05475         raise GwrfError("最大树深必须位于 1 到 200 之间，或留空表示不限制。")
05476     if min_samples_split < 2:
05477         raise GwrfError("最小分裂样本数必须大于或等于 2。")
05478     coordinates = prepared[[x_column, y_column]].to_numpy(dtype=float)
05479     _validate_coordinates(coordinates, coordinate_type)
05480     x = prepared[independent_columns]
05481     y = prepared[dependent_column]
05482     if (
05483         not np.isfinite(x.to_numpy(dtype=float)).all()
05484         or not np.isfinite(y.to_numpy(dtype=float)).all()
05485     ):
05486         raise GwrfError("所选变量中包含无穷值。")
05487     if y.nunique() < 2:
05488         raise GwrfError("因变量为常量，无法拟合 GWRF。")
05489     for column in independent_columns:
05490         if x[column].nunique() < 2:
05491             raise GwrfError(f"自变量“{column}”为常量。")
05492     interaction_columns = list(dict.fromkeys(shap_interaction_columns or []))
05493     if calculate_shap_interactions and not calculate_shap:
05494         raise GwrfError("计算 SHAP 交互效应前必须先启用 SHAP。")
05495     if calculate_shap_interactions:
05496         invalid_interaction_columns = [
05497             column

```

```

05498         for column in interaction_columns
05499         if column not in independent_columns
05500     ]
05501     if invalid_interaction_columns:
05502         raise GwrfError(
05503             "SHAP 交互变量必须来自已选择的自变量: "
05504             + "、".join(invalid_interaction_columns)
05505         )
05506     if len(interaction_columns) < 2:
05507         raise GwrfError("计算 SHAP 交互效应至少需要选择两个变量。")
05508     parameters: dict[str, int | None] = {
05509         "n_estimators": n_estimators,
05510         "max_depth": max_depth,
05511         "min_samples_split": min_samples_split,
05512     }
05513     if optimize_parameters:
05514         parameters, _ = _grid_search_parameters(
05515             x,
05516             y,
05517             n_estimators=n_estimators,
05518             max_depth=max_depth,
05519             min_samples_split=min_samples_split,
05520         )
05521     distance_coordinates = _neighbor_coordinates(coordinates, coordinate_type)
05522     bandwidth_search: list[dict[str, object]] = []
05523     if optimize_bandwidth:
05524         bandwidth, bandwidth_search = _select_bandwidth(
05525             x,
05526             y,
05527             distance_coordinates,
05528             candidates=bandwidth_candidates or [],
05529             parameters=parameters,
05530         )
05531     if min_samples_split > bandwidth:
05532         raise GwrfError("最小分裂样本数不能大于最终带宽。")
05533     query_count = bandwidth if fit_method == "in_sample" else bandwidth + 1
05534     distances, indices = (
05535         NearestNeighbors(n_neighbors=query_count)
05536         .fit(distance_coordinates)
05537         .kneighbors(distance_coordinates)
05538     )
05539     predictions: list[float] = []
05540     local_r_squared: list[float | None] = []
05541     shap_rows: list[np.ndarray] = []
05542     interaction_rows: list[dict[str, float]] = []
05543     importance_rows: list[dict[str, float]] = []
05544     rng = np.random.default_rng(RANDOM_SEED)
05545     shap_module = None
05546     if calculate_shap:
05547         import shap as shap_module
05548     for row_index in range(len(prepared)):
05549         local_indices = indices[row_index]
05550         local_distances = distances[row_index]
05551         if fit_method == "loocv":

```

```

05552     keep = local_indices != row_index
05553     local_indices = local_indices[keep][:bandwidth]
05554     local_distances = local_distances[keep][:bandwidth]
05555     weights = _bisquare(local_distances)
05556     if np.count_nonzero(weights) < 2:
05557         weights = np.ones_like(weights)
05558     local_x = x.iloc[local_indices]
05559     local_y = y.iloc[local_indices]
05560     model = RandomForestRegressor(
05561         n_estimators=int(parameters["n_estimators"]),
05562         max_depth=parameters["max_depth"],
05563         min_samples_split=int(parameters["min_samples_split"]),
05564         random_state=RANDOM_SEED,
05565         bootstrap=True,
05566         n_jobs=-1,
05567     )
05568     model.fit(local_x, local_y, sample_weight=weights)
05569     target = x.iloc[[row_index]]
05570     prediction = float(model.predict(target)[0])
05571     predictions.append(prediction)
05572     local_predictions = model.predict(local_x)
05573     local_mean = float(np.average(local_y, weights=weights))
05574     residual_sum = float(np.sum(weights * np.square(local_y - local_predictions)))
05575     total_sum = float(np.sum(weights * np.square(local_y - local_mean)))
05576     local_r_squared.append(None if total_sum <= 0 else 1 - residual_sum / total_sum)
05577     base_mse = mean_squared_error(local_y, local_predictions, sample_weight=weights)
05578     importance: dict[str, float] = {}
05579     for column in independent_columns:
05580         permuted = local_x.copy()
05581         permuted[column] = rng.permutation(permuted[column].to_numpy())
05582         permuted_mse = mean_squared_error(
05583             local_y, model.predict(permuted), sample_weight=weights
05584         )
05585         importance[column] = float(permuted_mse - base_mse)
05586     importance_rows.append(importance)
05587     if shap_module is not None:
05588         explainer = shap_module.TreeExplainer(model)
05589         explanation = explainer(target)
05590         shap_rows.append(np.asarray(explanation.values).reshape(-1))
05591     if calculate_shap_interactions:
05592         interaction_values = np.asarray(
05593             explainer.shap_interaction_values(target)
05594         )
05595         if interaction_values.ndim == 4:
05596             interaction_values = interaction_values[..., 0]
05597         interaction_matrix = interaction_values[0]
05598         interactions: dict[str, float] = {}
05599         for left_index, left_column in enumerate(interaction_columns):
05600             for right_column in interaction_columns[left_index + 1 :]:
05601                 x_left = independent_columns.index(left_column)
05602                 x_right = independent_columns.index(right_column)
05603                 interactions[f"{left_column}_{right_column}"] = float(
05604                     interaction_matrix[x_left, x_right]
05605                 )

```

```

05606         interaction_rows.append(interactions)
05607     prediction_array = np.asarray(predictions)
05608     residuals = y.to_numpy(dtype=float) - prediction_array
05609     rmse = math.sqrt(mean_squared_error(y, prediction_array))
05610     pseudo_r_squared = r2_score(y, prediction_array)
05611     importance_frame = pd.DataFrame(importance_rows)
05612     mean_importance = importance_frame.mean(axis=0)
05613     positive_importance = mean_importance.clip(lower=0)
05614     importance_total = float(positive_importance.sum())
05615     relative_importance = (
05616         positive_importance / importance_total * 100
05617         if importance_total > 0
05618         else positive_importance
05619     )
05620     importance_summary = [
05621         {
05622             "variable": column,
05623             "mean_permutation_importance": safe_float(mean_importance[column]),
05624             "relative_importance": safe_float(relative_importance[column]),
05625         }
05626         for column in relative_importance.sort_values(ascending=False).index
05627     ]
05628     shap_array = np.vstack(shap_rows) if shap_rows else None
05629     result_limit = (
05630         len(prepared) if include_full_local_results else min(100, len(prepared))
05631     )
05632     local_results: list[dict[str, object]] = []
05633     for row_index in range(result_limit):
05634         row: dict[str, object] = {
05635             "source_row": int(prepared.iloc[row_index]["__source_row__"]),
05636             "x_column": safe_float(coordinates[row_index, 0]),
05637             "y_column": safe_float(coordinates[row_index, 1]),
05638             "observed": safe_float(y.iloc[row_index]),
05639             "predicted": safe_float(prediction_array[row_index]),
05640             "residual": safe_float(residuals[row_index]),
05641             "local_r_squared": safe_float(local_r_squared[row_index]),
05642         }
05643         for column_index, column in enumerate(independent_columns):
05644             row[column] = safe_float(x.iloc[row_index][column])
05645             row[f"importance_{column}"] = safe_float(
05646                 importance_frame.iloc[row_index][column]
05647             )
05648             if shap_array is not None:
05649                 row[f"shap_{column}"] = safe_float(shap_array[row_index, column_index])
05650             if calculate_shap_interactions:
05651                 for pair, value in interaction_rows[row_index].items():
05652                     row[f"shap_interaction_{pair}"] = safe_float(value)
05653             local_results.append(row)
05654     return {
05655         "method": "gwr",
05656         "fit_method": fit_method,
05657         "observations": len(prepared),
05658         "dropped_rows": dropped_rows,
05659         "coordinate_type": coordinate_type,

```

```

05660 "x_column": x_column,
05661 "y_column": y_column,
05662 "dependent_column": dependent_column,
05663 "independent_columns": independent_columns,
05664 "bandwidth": bandwidth,
05665 "bandwidth_optimized": optimize_bandwidth,
05666 "bandwidth_search": bandwidth_search,
05667 "shap_calculated": calculate_shap,
05668 "shap_interactions_calculated": calculate_shap_interactions,
05669 "shap_interaction_columns": interaction_columns,
05670 "rf_parameters": parameters,
05671 "parameters_optimized": optimize_parameters,
05672 "metrics": {
05673     "pseudo_r_squared": safe_float(pseudo_r_squared),
05674     "rmse": safe_float(rmse),
05675 },
05676 "residual_moran": _residual_moran(residuals, distance_coordinates),
05677 "importance_summary": importance_summary,
05678 "local_result_count": len(prepared),
05679 "local_preview": local_results,
05680 }
05681 # Source file: backend/src/backend/reporting.py
05682 from io import BytesIO
05683 import pandas as pd
05684 def analysis_tables(result: dict[str, object]) -> tuple[pd.DataFrame, pd.DataFrame]:
05685     summary_rows = [
05686         {"item": "method", "value": result["method"]},
05687         {"item": "observations", "value": result["observations"]},
05688         {"item": "dropped_rows", "value": result["dropped_rows"]},
05689         {"item": "dependent_column", "value": result["dependent_column"]},
05690         {
05691             "item": "independent_columns",
05692             "value": ", ".join(result["independent_columns"]),
05693         },
05694     ]
05695     correlation = result.get("correlation")
05696     if correlation:
05697         summary_rows.extend(
05698             {"item": key, "value": value} for key, value in correlation.items()
05699         )
05700         detail = pd.DataFrame([correlation])
05701     else:
05702         regression = result["regression"]
05703         summary_rows.extend(
05704             {"item": key, "value": value}
05705             for key, value in regression["metrics"].items()
05706         )
05707         detail = pd.DataFrame(regression["coefficients"])
05708     diagnostics = result.get("diagnostics")
05709     if diagnostics:
05710         summary_rows.extend(
05711             {"item": key, "value": diagnostics[key]}
05712             for key in (
05713                 "converged",

```

```

05714         "valid_inference",
05715         "rank",
05716         "parameter_count",
05717         "condition_number",
05718         "raw_condition_number",
05719         "scale_ratio",
05720         "max_vif",
05721     )
05722 )
05723 summary_rows.extend(
05724     {"item": f"warning_{index}", "value": message}
05725     for index, message in enumerate(diagnostics["warnings"], start=1)
05726 )
05727 return pd.DataFrame(summary_rows), detail
05728 def export_analysis_csv(result: dict[str, object]) -> bytes:
05729     _, detail = analysis_tables(result)
05730     return detail.to_csv(index=False, lineterminator="\n").encode("utf-8-sig")
05731 def export_analysis_xlsx(result: dict[str, object]) -> bytes:
05732     summary, detail = analysis_tables(result)
05733     output = BytesIO()
05734     with pd.ExcelWriter(output, engine="openpyxl") as writer:
05735         summary.to_excel(writer, sheet_name="summary", index=False)
05736         detail.to_excel(writer, sheet_name="coefficients", index=False)
05737         diagnostics = result.get("diagnostics")
05738         if diagnostics and diagnostics["vif"]:
05739             pd.DataFrame(diagnostics["vif"]).to_excel(
05740                 writer, sheet_name="vif", index=False
05741             )
05742     return output.getvalue()
05743 def spatial_tables(result: dict[str, object]) -> dict[str, pd.DataFrame]:
05744     weights = result["weights"]
05745     summary_rows = [
05746         {"item": "method", "value": result["method"]},
05747         {"item": "observations", "value": result["observations"]},
05748         {"item": "dropped_rows", "value": result["dropped_rows"]},
05749         {"item": "coordinate_type", "value": result["coordinate_type"]},
05750         {"item": "x_column", "value": result["x_column"]},
05751         {"item": "y_column", "value": result["y_column"]},
05752         {"item": "dependent_column", "value": result["dependent_column"]},
05753         {
05754             "item": "independent_columns",
05755             "value": ", ".join(result["independent_columns"]),
05756         },
05757         {"item": "weights_type", "value": weights["type"]},
05758         {"item": "neighbors", "value": weights["neighbors"]},
05759         {"item": "weights_transformation", "value": weights["transformation"]},
05760         {"item": "connected_components", "value": weights["components"]},
05761     ]
05762     tables: dict[str, pd.DataFrame] = {}
05763     if result.get("morán"):
05764         summary_rows.extend(
05765             {"item": key, "value": value} for key, value in result["morán"].items()
05766         )
05767     tables["morán"] = pd.DataFrame(

```

```

05768         [{"item": key, "value": value} for key, value in result["moran"].items()]
05769     )
05770     elif result.get("regression"):
05771         regression = result["regression"]
05772         summary_rows.extend(
05773             {"item": key, "value": value}
05774             for key, value in regression["metrics"].items()
05775         )
05776         tables["coefficients"] = pd.DataFrame(regression["coefficients"])
05777         if regression["spatial_impacts"]:
05778             tables["spatial_impacts"] = pd.DataFrame(regression["spatial_impacts"])
05779     else:
05780         gwr = result["gwr"]
05781         summary_rows.append({"item": "bandwidth", "value": gwr["bandwidth"]})
05782         summary_rows.extend(
05783             {"item": key, "value": value} for key, value in gwr["metrics"].items()
05784         )
05785         tables["coefficient_summary"] = pd.DataFrame(gwr["coefficient_summaries"])
05786         tables["local_results"] = pd.DataFrame(gwr["local_preview"])
05787     residual_moran = result.get("residual_moran")
05788     if residual_moran:
05789         summary_rows.extend(
05790             {"item": f"residual_moran_{key}", "value": value}
05791             for key, value in residual_moran.items()
05792         )
05793         tables["residual_moran"] = pd.DataFrame(
05794             [{"item": key, "value": value} for key, value in residual_moran.items()]
05795         )
05796     model_selection = result.get("model_selection")
05797     if model_selection:
05798         summary_rows.append(
05799             {
05800                 "item": "model_selection_recommendation",
05801                 "value": model_selection["recommendation"],
05802             }
05803         )
05804         baseline_moran = model_selection["baseline_residual_moran"]
05805         if baseline_moran:
05806             summary_rows.extend(
05807                 {"item": f"baseline_residual_moran_{key}", "value": value}
05808                 for key, value in baseline_moran.items()
05809             )
05810         selection_rows = list(model_selection["tests"])
05811         if selection_rows:
05812             selection_table = pd.DataFrame(selection_rows)
05813             selection_table["recommendation"] = model_selection["recommendation"]
05814             tables["model_selection"] = selection_table
05815         diagnostics = result["diagnostics"]
05816         diagnostic_rows = [
05817             {"item": key, "value": diagnostics[key]}
05818             for key in (
05819                 "converged",
05820                 "valid_inference",
05821                 "rank",

```

```

05822         "parameter_count",
05823         "condition_number",
05824         "raw_condition_number",
05825         "scale_ratio",
05826         "max_vif",
05827     )
05828 ]
05829 diagnostic_rows.extend(
05830     {"item": f"warning_{index}", "value": message}
05831     for index, message in enumerate(diagnostics["warnings"], start=1)
05832 )
05833 tables["summary"] = pd.DataFrame(summary_rows)
05834 tables["diagnostics"] = pd.DataFrame(diagnostic_rows)
05835 if diagnostics["vif"]:
05836     tables["vif"] = pd.DataFrame(diagnostics["vif"])
05837 return tables
05838 def export_spatial_csv(result: dict[str, object]) -> bytes:
05839     tables = spatial_tables(result)
05840     if "local_results" in tables:
05841         selected = tables["local_results"]
05842     elif "coefficients" in tables:
05843         selected = tables["coefficients"]
05844     elif "moran" in tables:
05845         selected = tables["moran"]
05846     else:
05847         selected = tables["summary"]
05848     diagnostics = result["diagnostics"]
05849     weights = result["weights"]
05850     metadata = {
05851         "report_method": result["method"],
05852         "report_observations": result["observations"],
05853         "report_dropped_rows": result["dropped_rows"],
05854         "report_coordinate_type": result["coordinate_type"],
05855         "report_neighbors": weights["neighbors"],
05856         "report_converged": diagnostics["converged"],
05857         "report_valid_inference": diagnostics["valid_inference"],
05858         "report_condition_number": diagnostics["condition_number"],
05859         "report_raw_condition_number": diagnostics["raw_condition_number"],
05860         "report_max_vif": diagnostics["max_vif"],
05861         "report_warnings": " | ".join(diagnostics["warnings"]),
05862     }
05863     residual_moran = result.get("residual_moran")
05864     if residual_moran:
05865         metadata.update(
05866             {
05867                 "report_residual_moran_i": residual_moran["i"],
05868                 "report_residual_moran_p_permutation": residual_moran["p_permutation"],
05869             }
05870         )
05871     model_selection = result.get("model_selection")
05872     if model_selection:
05873         metadata["report_model_recommendation"] = model_selection["recommendation"]
05874     exported = selected.copy()
05875     for column, value in reversed(metadata.items()):

```

```

05876     exported.insert(0, column, value)
05877     return exported.to_csv(index=False, lineterminator="\n").encode("utf-8-sig")
05878 def export_spatial_xlsx(result: dict[str, object]) -> bytes:
05879     output = BytesIO()
05880     with pd.ExcelWriter(output, engine="openpyxl") as writer:
05881         for sheet_name, table in spatial_tables(result).items():
05882             table.to_excel(writer, sheet_name=sheet_name[:31], index=False)
05883     return output.getvalue()
05884 def gwr_tables(result: dict[str, object]) -> dict[str, pd.DataFrame]:
05885     parameters = result["rf_parameters"]
05886     metrics = result["metrics"]
05887     summary_rows = [
05888         {"item": "method", "value": result["method"]},
05889         {"item": "fit_method", "value": result["fit_method"]},
05890         {"item": "observations", "value": result["observations"]},
05891         {"item": "dropped_rows", "value": result["dropped_rows"]},
05892         {"item": "coordinate_type", "value": result["coordinate_type"]},
05893         {"item": "x_column", "value": result["x_column"]},
05894         {"item": "y_column", "value": result["y_column"]},
05895         {"item": "dependent_column", "value": result["dependent_column"]},
05896         {
05897             "item": "independent_columns",
05898             "value": ", ".join(result["independent_columns"]),
05899         },
05900         {"item": "bandwidth", "value": result["bandwidth"]},
05901         {"item": "bandwidth_optimized", "value": result["bandwidth_optimized"]},
05902         {"item": "shap_calculated", "value": result["shap_calculated"]},
05903         {
05904             "item": "shap_interactions_calculated",
05905             "value": result["shap_interactions_calculated"],
05906         },
05907         {
05908             "item": "shap_interaction_columns",
05909             "value": ", ".join(result["shap_interaction_columns"]),
05910         },
05911         {"item": "parameters_optimized", "value": result["parameters_optimized"]},
05912         {"item": "n_estimators", "value": parameters["n_estimators"]},
05913         {"item": "max_depth", "value": parameters["max_depth"]},
05914         {"item": "min_samples_split", "value": parameters["min_samples_split"]},
05915         {"item": "pseudo_r_squared", "value": metrics["pseudo_r_squared"]},
05916         {"item": "rmse", "value": metrics["rmse"]},
05917     ]
05918     summary_rows.extend(
05919         {"item": f"residual_moran_{key}", "value": value}
05920         for key, value in result["residual_moran"].items()
05921     )
05922     tables = {
05923         "summary": pd.DataFrame(summary_rows),
05924         "relative_importance": pd.DataFrame(result["importance_summary"]),
05925         "local_results": pd.DataFrame(result["local_preview"]),
05926         "residual_moran": pd.DataFrame(
05927             [
05928                 {"item": key, "value": value}
05929                 for key, value in result["residual_moran"].items()

```

```

05930     ]
05931     ),
05932 }
05933 if result["bandwidth_search"]:
05934     tables["bandwidth_search"] = pd.DataFrame(result["bandwidth_search"])
05935 return tables
05936 def export_gwrf_csv(result: dict[str, object]) -> bytes:
05937     return (
05938         pd.DataFrame(result["local_preview"])
05939         .to_csv(index=False, lineterminator="\n")
05940         .encode("utf-8-sig")
05941     )
05942 def export_gwrf_xlsx(result: dict[str, object]) -> bytes:
05943     output = BytesIO()
05944     with pd.ExcelWriter(output, engine="openpyxl") as writer:
05945         for sheet_name, table in gwrf_tables(result).items():
05946             table.to_excel(writer, sheet_name=sheet_name[:31], index=False)
05947     return output.getvalue()
05948 # Source file: backend/src/backend/main.py
05949 import json
05950 import logging
05951 from io import BytesIO
05952 from pathlib import Path
05953 from threading import Lock
05954 from typing import Annotated
05955 from urllib.parse import quote
05956 from uuid import uuid4
05957 from zipfile import BadZipFile
05958 import pandas as pd
05959 from fastapi import FastAPI, File, Form, HTTPException, UploadFile, status
05960 from fastapi.responses import Response
05961 from fastapi.staticfiles import StaticFiles
05962 from backend.analysis import AnalysisError, AnalysisMethod, run_analysis
05963 from backend.cleaning import (
05964     ExportFormat,
05965     ExportTable,
05966     MissingAction,
05967     StandardizationError,
05968     StandardizationMethod,
05969     clean_dataframe,
05970     export_csv,
05971     export_xlsx,
05972 )
05973 from backend.gwrf import (
05974     GwrfError,
05975     GwrfFitMethod,
05976     optimize_gwrf_bandwidth,
05977     optimize_gwrf_parameters,
05978     run_gwrf,
05979 )
05980 from backend.quality import build_quality_report
05981 from backend.reporting import (
05982     export_analysis_csv,
05983     export_analysis_xlsx,

```

```

05984     export_gwrf_csv,
05985     export_gwrf_xlsx,
05986     export_spatial_csv,
05987     export_spatial_xlsx,
05988 )
05989 from backend.resources import frontend_directory
05990 from backend.spatial import (
05991     CoordinateType,
05992     SpatialAnalysisError,
05993     SpatialMethod,
05994     run_spatial_analysis,
05995 )
05996 app = FastAPI(
05997     title="Data Analysis Desktop API",
05998     version="1.0.0",
05999 )
06000 LOGGER = logging.getLogger(__name__)
06001 _GWRF_EXPORT_CACHE_LOCK = Lock()
06002 _GWRF_EXPORT_CACHE: dict[str, tuple[str, dict[str, object]]] = {}
06003 MAX_UPLOAD_BYTES = 20 * 1024 * 1024
06004 SUPPORTED_FILE_TYPES = {".csv", ".xlsx"}
06005 def cache_gwrf_export_result(filename: str, result: dict[str, object]) -> str:
06006     """Keep only the latest full GWRF result for the desktop export workflow."""
06007     export_id = uuid4().hex
06008     with _GWRF_EXPORT_CACHE_LOCK:
06009         _GWRF_EXPORT_CACHE.clear()
06010         _GWRF_EXPORT_CACHE[export_id] = (filename, result)
06011     return export_id
06012 def get_gwrf_export_result(export_id: str) -> tuple[str, dict[str, object]] | None:
06013     with _GWRF_EXPORT_CACHE_LOCK:
06014         return _GWRF_EXPORT_CACHE.get(export_id)
06015 @app.get("/health")
06016 def health_check() -> dict[str, str]:
06017     return {"status": "ok"}
06018 def read_dataframe(filename: str, content: bytes) -> pd.DataFrame:
06019     suffix = Path(filename).suffix.lower()
06020     try:
06021         if suffix == ".csv":
06022             for encoding in ("utf-8-sig", "gb18030"):
06023                 try:
06024                     return pd.read_csv(BytesIO(content), encoding=encoding)
06025                 except UnicodeDecodeError:
06026                     continue
06027             raise HTTPException(
06028                 status_code=status.HTTP_400_BAD_REQUEST,
06029                 detail="CSV encoding is not supported. Use UTF-8 or GB18030.",
06030             )
06031         return pd.read_excel(BytesIO(content), engine="openpyxl")
06032     except (BadZipFile, KeyError, ValueError, pd.errors.ParserError) as exc:
06033         raise HTTPException(
06034             status_code=status.HTTP_400_BAD_REQUEST,
06035             detail="The uploaded file could not be parsed.",
06036         ) from exc
06037 async def load_uploaded_dataframe(file: UploadFile) -> tuple[str, pd.DataFrame]:

```

```

06038 filename = file.filename or ""
06039 suffix = Path(filename).suffix.lower()
06040 if suffix not in SUPPORTED_FILE_TYPES:
06041     raise HTTPException(
06042         status_code=status.HTTP_400_BAD_REQUEST,
06043         detail="Only CSV and XLSX files are supported.",
06044     )
06045 content = await file.read(MAX_UPLOAD_BYTES + 1)
06046 await file.close()
06047 if not content:
06048     raise HTTPException(
06049         status_code=status.HTTP_400_BAD_REQUEST,
06050         detail="The uploaded file is empty.",
06051     )
06052 if len(content) > MAX_UPLOAD_BYTES:
06053     raise HTTPException(
06054         status_code=status.HTTP_413_CONTENT_TOO_LARGE,
06055         detail="The uploaded file exceeds the 20 MB limit.",
06056     )
06057 return filename, read_dataframe(filename, content)
06058 def dataframe_preview(dataframe: pd.DataFrame) -> list[dict[str, object]]:
06059     return json.loads(dataframe.head(100).to_json(orient="records", date_format="iso"))
06060 def build_download_headers(filename: str) -> dict[str, str]:
06061     fallback = "data-dataex" + Path(filename).suffix.lower()
06062     encoded = quote(filename)
06063     return {
06064         "Content-Disposition": (
06065             f"attachment; filename=\"{fallback}\"; filename*=UTF-8\"{encoded}\"
06066         )
06067     }
06068 def parse_standardization_columns(raw_columns: str) -> list[str]:
06069     try:
06070         columns = json.loads(raw_columns)
06071     except json.JSONDecodeError as exc:
06072         raise HTTPException(
06073             status_code=status.HTTP_400_BAD_REQUEST,
06074             detail="Standardization columns must be a JSON array.",
06075         ) from exc
06076     if not isinstance(columns, list) or not all(
06077         isinstance(column, str) for column in columns
06078     ):
06079         raise HTTPException(
06080             status_code=status.HTTP_400_BAD_REQUEST,
06081             detail="Standardization columns must be a JSON array of column names.",
06082         )
06083     return columns
06084 def parse_analysis_columns(raw_columns: str) -> list[str]:
06085     try:
06086         columns = json.loads(raw_columns)
06087     except json.JSONDecodeError as exc:
06088         raise HTTPException(
06089             status_code=status.HTTP_400_BAD_REQUEST,
06090             detail="Independent variables must be a JSON array.",
06091         ) from exc

```

```

06092     if not isinstance(columns, list) or not all(
06093         isinstance(column, str) for column in columns
06094     ):
06095         raise HTTPException(
06096             status_code=status.HTTP_400_BAD_REQUEST,
06097             detail="Independent variables must be a JSON array of column names.",
06098         )
06099     return columns
06100 def parse_bandwidth_candidates(raw_candidates: str) -> list[int]:
06101     try:
06102         candidates = json.loads(raw_candidates)
06103     except json.JSONDecodeError as exc:
06104         raise HTTPException(
06105             status_code=status.HTTP_400_BAD_REQUEST,
06106             detail="候选带宽必须是 JSON 整数数组。",
06107         ) from exc
06108     if not isinstance(candidates, list) or not all(
06109         isinstance(candidate, int) and not isinstance(candidate, bool)
06110         for candidate in candidates
06111     ):
06112         raise HTTPException(
06113             status_code=status.HTTP_400_BAD_REQUEST,
06114             detail="候选带宽必须是 JSON 整数数组。",
06115         )
06116     return candidates
06117 @app.post("/api/files/preview")
06118 async def preview_file(
06119     file: Annotated[UploadFile, File()],
06120 ) -> dict[str, object]:
06121     filename, dataframe = await load_uploaded_dataframe(file)
06122     return {
06123         "filename": filename,
06124         "row_count": int(dataframe.shape[0]),
06125         "column_count": int(dataframe.shape[1]),
06126         "columns": [str(column) for column in dataframe.columns],
06127         "preview": dataframe_preview(dataframe),
06128         "quality": build_quality_report(dataframe),
06129     }
06130 @app.post("/api/files/clean/preview")
06131 async def preview_cleaning(
06132     file: Annotated[UploadFile, File()],
06133     missing_action: Annotated[MissingAction, Form()] = "none",
06134     trim_whitespace: Annotated[bool, Form()] = False,
06135     remove_line_breaks: Annotated[bool, Form()] = False,
06136     standardization_method: Annotated[StandardizationMethod, Form()] = "none",
06137     standardization_columns: Annotated[str, Form()] = "",
06138 ) -> dict[str, object]:
06139     filename, dataframe = await load_uploaded_dataframe(file)
06140     try:
06141         cleaned, extracted, summary = clean_dataframe(
06142             dataframe,
06143             missing_action=missing_action,
06144             trim_whitespace=trim_whitespace,
06145             remove_line_breaks=remove_line_breaks,

```

```

06146         standardization_method=standardization_method,
06147         standardization_columns=parse_standardization_columns(
06148             standardization_columns
06149         ),
06150     )
06151 except StandardizationError as exc:
06152     raise HTTPException(
06153         status_code=status.HTTP_400_BAD_REQUEST,
06154         detail=str(exc),
06155     ) from exc
06156 return {
06157     "filename": filename,
06158     "original_row_count": int(dataframe.shape[0]),
06159     "cleaned_row_count": int(cleaned.shape[0]),
06160     "extracted_row_count": int(extracted.shape[0]),
06161     "columns": [str(column) for column in dataframe.columns],
06162     "cleaned_preview": dataframe_preview(cleaned),
06163     "extracted_preview": dataframe_preview(extracted),
06164     "cleaned_quality": build_quality_report(cleaned),
06165     "summary": summary,
06166 }
06167 @app.post("/api/files/clean/export")
06168 async def export_cleaning(
06169     file: Annotated[UploadFile, File()],
06170     missing_action: Annotated[MissingAction, Form()] = "none",
06171     trim_whitespace: Annotated[bool, Form()] = False,
06172     remove_line_breaks: Annotated[bool, Form()] = False,
06173     output_format: Annotated[ExportFormat, Form()] = "xlsx",
06174     table: Annotated[ExportTable, Form()] = "cleaned",
06175     standardization_method: Annotated[StandardizationMethod, Form()] = "none",
06176     standardization_columns: Annotated[str, Form()] = "",
06177 ) -> Response:
06178     filename, dataframe = await load_uploaded_dataframe(file)
06179     try:
06180         cleaned, extracted, _ = clean_dataframe(
06181             dataframe,
06182             missing_action=missing_action,
06183             trim_whitespace=trim_whitespace,
06184             remove_line_breaks=remove_line_breaks,
06185             standardization_method=standardization_method,
06186             standardization_columns=parse_standardization_columns(
06187                 standardization_columns
06188             ),
06189         )
06190     except StandardizationError as exc:
06191         raise HTTPException(
06192             status_code=status.HTTP_400_BAD_REQUEST,
06193             detail=str(exc),
06194         ) from exc
06195     stem = Path(filename).stem
06196     if output_format == "xlsx":
06197         download_name = f"{stem}-dataex.xlsx"
06198         return Response(
06199             content=export_xlsx(cleaned, extracted),

```

```

06200     media_type=(
06201         "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
06202     ),
06203     headers=build_download_headers(download_name),
06204 )
06205 selected = cleaned if table == "cleaned" else extracted
06206 qualifier = "" if table == "cleaned" else "-empty"
06207 download_name = f"{stem}{qualifier}-dataex.csv"
06208 return Response(
06209     content=export_csv(selected),
06210     media_type="text/csv; charset=utf-8",
06211     headers=build_download_headers(download_name),
06212 )
06213 @app.post("/api/analysis/run")
06214 async def analyze_file(
06215     file: Annotated[UploadFile, File()],
06216     method: Annotated[AnalysisMethod, Form()],
06217     dependent_column: Annotated[str, Form()],
06218     independent_columns: Annotated[str, Form()],
06219 ) -> dict[str, object]:
06220     _, dataframe = await load_uploaded_dataframe(file)
06221     try:
06222         return run_analysis(
06223             dataframe,
06224             method=method,
06225             dependent_column=dependent_column,
06226             independent_columns=parse_analysis_columns(independent_columns),
06227         )
06228     except AnalysisError as exc:
06229         raise HTTPException(
06230             status_code=status.HTTP_400_BAD_REQUEST,
06231             detail=str(exc),
06232         ) from exc
06233 @app.post("/api/analysis/export")
06234 async def export_analysis(
06235     file: Annotated[UploadFile, File()],
06236     method: Annotated[AnalysisMethod, Form()],
06237     dependent_column: Annotated[str, Form()],
06238     independent_columns: Annotated[str, Form()],
06239     output_format: Annotated[ExportFormat, Form()] = "xlsx",
06240 ) -> Response:
06241     filename, dataframe = await load_uploaded_dataframe(file)
06242     try:
06243         result = run_analysis(
06244             dataframe,
06245             method=method,
06246             dependent_column=dependent_column,
06247             independent_columns=parse_analysis_columns(independent_columns),
06248         )
06249     except AnalysisError as exc:
06250         raise HTTPException(
06251             status_code=status.HTTP_400_BAD_REQUEST,
06252             detail=str(exc),
06253         ) from exc

```

```

06254 stem = Path(filename).stem
06255 download_name = f"{stem}-analysis-dataex.{output_format}"
06256 if output_format == "xlsx":
06257     content = export_analysis_xlsx(result)
06258     media_type = "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
06259 else:
06260     content = export_analysis_csv(result)
06261     media_type = "text/csv; charset=utf-8"
06262 return Response(
06263     content=content,
06264     media_type=media_type,
06265     headers=build_download_headers(download_name),
06266 )
06267 @app.post("/api/spatial/run")
06268 async def analyze_spatial_file(
06269     file: Annotated[UploadFile, File()],
06270     method: Annotated[SpatialMethod, Form()],
06271     coordinate_type: Annotated[CoordinateType, Form()],
06272     x_column: Annotated[str, Form()],
06273     y_column: Annotated[str, Form()],
06274     dependent_column: Annotated[str, Form()],
06275     independent_columns: Annotated[str, Form()] = "",
06276     neighbors: Annotated[int, Form()] = 8,
06277 ) -> dict[str, object]:
06278     _, dataframe = await load_uploaded_dataframe(file)
06279     try:
06280         return run_spatial_analysis(
06281             dataframe,
06282             method=method,
06283             coordinate_type=coordinate_type,
06284             x_column=x_column,
06285             y_column=y_column,
06286             dependent_column=dependent_column,
06287             independent_columns=parse_analysis_columns(independent_columns),
06288             neighbors=neighbors,
06289         )
06290     except SpatialAnalysisError as exc:
06291         raise HTTPException(
06292             status_code=status.HTTP_400_BAD_REQUEST,
06293             detail=str(exc),
06294         ) from exc
06295 @app.post("/api/spatial/export")
06296 async def export_spatial_analysis(
06297     file: Annotated[UploadFile, File()],
06298     method: Annotated[SpatialMethod, Form()],
06299     coordinate_type: Annotated[CoordinateType, Form()],
06300     x_column: Annotated[str, Form()],
06301     y_column: Annotated[str, Form()],
06302     dependent_column: Annotated[str, Form()],
06303     independent_columns: Annotated[str, Form()] = "",
06304     neighbors: Annotated[int, Form()] = 8,
06305     output_format: Annotated[ExportFormat, Form()] = "xlsx",
06306 ) -> Response:
06307     filename, dataframe = await load_uploaded_dataframe(file)

```

```

06308 try:
06309     result = run_spatial_analysis(
06310         dataframe,
06311         method=method,
06312         coordinate_type=coordinate_type,
06313         x_column=x_column,
06314         y_column=y_column,
06315         dependent_column=dependent_column,
06316         independent_columns=parse_analysis_columns(independent_columns),
06317         neighbors=neighbors,
06318         include_full_local_results=True,
06319     )
06320 except SpatialAnalysisError as exc:
06321     raise HTTPException(
06322         status_code=status.HTTP_400_BAD_REQUEST,
06323         detail=str(exc),
06324     ) from exc
06325 stem = Path(filename).stem
06326 download_name = f"{stem}-spatial-dataex.{output_format}"
06327 if output_format == "xlsx":
06328     content = export_spatial_xlsx(result)
06329     media_type = "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
06330 else:
06331     content = export_spatial_csv(result)
06332     media_type = "text/csv; charset=utf-8"
06333 return Response(
06334     content=content,
06335     media_type=media_type,
06336     headers=build_download_headers(download_name),
06337 )
06338 @app.post("/api/gwrf/optimize-parameters")
06339 async def optimize_gwrf_file_parameters(
06340     file: Annotated[UploadFile, File()],
06341     dependent_column: Annotated[str, Form()],
06342     independent_columns: Annotated[str, Form()],
06343 ) -> dict[str, object]:
06344     _, dataframe = await load_uploaded_dataframe(file)
06345     try:
06346         return optimize_gwrf_parameters(
06347             dataframe,
06348             dependent_column=dependent_column,
06349             independent_columns=parse_analysis_columns(independent_columns),
06350         )
06351     except GwrfError as exc:
06352         raise HTTPException(
06353             status_code=status.HTTP_400_BAD_REQUEST,
06354             detail=str(exc),
06355         ) from exc
06356 @app.post("/api/gwrf/optimize-bandwidth")
06357 async def optimize_gwrf_file_bandwidth(
06358     file: Annotated[UploadFile, File()],
06359     coordinate_type: Annotated[CoordinateType, Form()],
06360     x_column: Annotated[str, Form()],
06361     y_column: Annotated[str, Form()],

```

```

06362     dependent_column: Annotated[str, Form()],
06363     independent_columns: Annotated[str, Form()],
06364     bandwidth_candidates: Annotated[str, Form()],
06365     n_estimators: Annotated[int, Form()] = 200,
06366     max_depth: Annotated[int | None, Form()] = 10,
06367     min_samples_split: Annotated[int, Form()] = 5,
06368 ) -> dict[str, object]:
06369     _, dataframe = await load_uploaded_dataframe(file)
06370     try:
06371         return optimize_gwrf_bandwidth(
06372             dataframe,
06373             coordinate_type=coordinate_type,
06374             x_column=x_column,
06375             y_column=y_column,
06376             dependent_column=dependent_column,
06377             independent_columns=parse_analysis_columns(independent_columns),
06378             bandwidth_candidates=parse_bandwidth_candidates(bandwidth_candidates),
06379             n_estimators=n_estimators,
06380             max_depth=None if max_depth == 0 else max_depth,
06381             min_samples_split=min_samples_split,
06382         )
06383     except GwrfError as exc:
06384         raise HTTPException(
06385             status_code=status.HTTP_400_BAD_REQUEST,
06386             detail=str(exc),
06387         ) from exc
06388 @app.post("/api/gwrf/run")
06389 async def analyze_gwrf_file(
06390     file: Annotated[UploadFile, File()],
06391     coordinate_type: Annotated[CoordinateType, Form()],
06392     x_column: Annotated[str, Form()],
06393     y_column: Annotated[str, Form()],
06394     dependent_column: Annotated[str, Form()],
06395     independent_columns: Annotated[str, Form()],
06396     bandwidth: Annotated[int, Form()],
06397     fit_method: Annotated[GwrfFitMethod, Form()] = "in_sample",
06398     n_estimators: Annotated[int, Form()] = 200,
06399     max_depth: Annotated[int | None, Form()] = 10,
06400     min_samples_split: Annotated[int, Form()] = 5,
06401     optimize_parameters: Annotated[bool, Form()] = False,
06402     optimize_bandwidth: Annotated[bool, Form()] = False,
06403     bandwidth_candidates: Annotated[str, Form()] = "",
06404     calculate_shap: Annotated[bool, Form()] = False,
06405     calculate_shap_interactions: Annotated[bool, Form()] = False,
06406     shap_interaction_columns: Annotated[str, Form()] = "",
06407 ) -> dict[str, object]:
06408     filename, dataframe = await load_uploaded_dataframe(file)
06409     try:
06410         result = run_gwrf(
06411             dataframe,
06412             coordinate_type=coordinate_type,
06413             x_column=x_column,
06414             y_column=y_column,
06415             dependent_column=dependent_column,

```

```

06416     independent_columns=parse_analysis_columns(independent_columns),
06417     bandwidth=bandwidth,
06418     fit_method=fit_method,
06419     n_estimators=n_estimators,
06420     max_depth=None if max_depth == 0 else max_depth,
06421     min_samples_split=min_samples_split,
06422     optimize_parameters=optimize_parameters,
06423     optimize_bandwidth=optimize_bandwidth,
06424     bandwidth_candidates=parse_bandwidth_candidates(bandwidth_candidates),
06425     calculate_shap=calculate_shap,
06426     calculate_shap_interactions=calculate_shap_interactions,
06427     shap_interaction_columns=parse_analysis_columns(shap_interaction_columns),
06428     include_full_local_results=True,
06429 )
06430 except GwrfError as exc:
06431     raise HTTPException(
06432         status_code=status.HTTP_400_BAD_REQUEST,
06433         detail=str(exc),
06434     ) from exc
06435 export_id = cache_gwrf_export_result(filename, result)
06436 local_results = result.get("local_preview", [])
06437 response_result = dict(result)
06438 response_result["local_preview"] = local_results[:100]
06439 response_result["export_id"] = export_id
06440 LOGGER.info(
06441     "GWRF result prepared for export: source=%s export_id=%s rows=%d columns=%d",
06442     filename,
06443     export_id,
06444     len(local_results),
06445     len(local_results[0]) if local_results else 0,
06446 )
06447 return response_result
06448 @app.post("/api/gwrf/export")
06449 async def export_gwrf_analysis(
06450     export_id: Annotated[str, Form()],
06451     output_format: Annotated[ExportFormat, Form()] = "xlsx",
06452 ) -> Response:
06453     cached = get_gwrf_export_result(export_id)
06454     if cached is None:
06455         raise HTTPException(
06456             status_code=status.HTTP_409_CONFLICT,
06457             detail="GWRF 完整结果已失效，请重新运行最终模型后再导出。",
06458         )
06459     filename, result = cached
06460     local_results = result.get("local_preview", [])
06461     row_count = len(local_results)
06462     column_count = len(local_results[0]) if local_results else 0
06463     stem = Path(filename).stem
06464     download_name = f"{stem}-gwrf-dataex.{output_format}"
06465     LOGGER.info(
06466         "GWRF export started: output_path=browser_download filename=%s rows=%d columns=%d",
06467         download_name,
06468         row_count,
06469         column_count,

```

```

06470 )
06471 try:
06472     if output_format == "xlsx":
06473         content = export_gwrf_xlsx(result)
06474         media_type = "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
06475     else:
06476         content = export_gwrf_csv(result)
06477         media_type = "text/csv; charset=utf-8"
06478     if not content:
06479         raise ValueError("生成的导出文件为空。")
06480 except Exception as exc:
06481     LOGGER.exception(
06482         "GWRF export failed: filename=%s rows=%d columns=%d",
06483         download_name,
06484         row_count,
06485         column_count,
06486     )
06487     raise HTTPException(
06488         status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
06489         detail=f"GWRF 结果文件生成失败: {exc}",
06490     ) from exc
06491     LOGGER.info(
06492         "GWRF export generated successfully: filename=%s rows=%d columns=%d bytes=%d",
06493         download_name,
06494         row_count,
06495         column_count,
06496         len(content),
06497     )
06498     return Response(
06499         content=content,
06500         media_type=media_type,
06501         headers=build_download_headers(download_name),
06502     )
06503 STATIC_DIRECTORY = frontend_directory()
06504 if STATIC_DIRECTORY.is_dir():
06505     app.mount("/", StaticFiles(directory=STATIC_DIRECTORY, html=True), name="frontend")
06506 # Source file: backend/src/backend/desktop.py
06507 import ctypes
06508 import logging
06509 import os
06510 import socket
06511 import sys
06512 import threading
06513 import time
06514 from pathlib import Path
06515 from urllib.error import URLError
06516 from urllib.request import urlopen
06517 import uvicorn
06518 import webview
06519 from backend.main import app
06520 from backend.resources import frontend_directory
06521 APP_NAME = "DataEX-G"
06522 APP_TITLE = "DataEX-G"
06523 MUTEX_NAME = "Local\\GGzzzW.DataEX-G"

```

```

06524  LOGGER = logging.getLogger(__name__)
06525  def local_data_directory() -> Path:
06526      base = Path(os.environ.get("LOCALAPPDATA", Path.home() / "AppData" / "Local"))
06527      directory = base / APP_NAME
06528      directory.mkdir(parents=True, exist_ok=True)
06529      return directory
06530  def configure_logging() -> None:
06531      log_directory = local_data_directory() / "logs"
06532      log_directory.mkdir(parents=True, exist_ok=True)
06533      logging.basicConfig(
06534          filename=log_directory / "desktop.log",
06535          level=logging.INFO,
06536          format="%(%asctime)s %(levelname)s %(name)s %(message)s",
06537          encoding="utf-8",
06538      )
06539  def show_error(message: str) -> None:
06540      LOGGER.error(message)
06541      if sys.platform == "win32":
06542          ctypes.windll.user32.MessageBoxW(0, message, APP_TITLE, 0x10)
06543      else:
06544          print(message, file=sys.stderr)
06545  def acquire_single_instance() -> int | None:
06546      if sys.platform != "win32":
06547          return 1
06548      handle = ctypes.windll.kernel32.CreateMutexW(None, False, MUTEX_NAME)
06549      if not handle:
06550          return None
06551      if ctypes.windll.kernel32.GetLastError() == 183:
06552          ctypes.windll.kernel32.CloseHandle(handle)
06553      return None
06554      return int(handle)
06555  def release_single_instance(handle: int | None) -> None:
06556      if sys.platform == "win32" and handle:
06557          ctypes.windll.kernel32.CloseHandle(handle)
06558  def create_server() -> tuple[uvicorn.Server, socket.socket, int]:
06559      server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
06560      server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
06561      server_socket.bind(("127.0.0.1", 0))
06562      server_socket.listen(2048)
06563      port = int(server_socket.getsockname()[1])
06564      config = uvicorn.Config(
06565          app,
06566          host="127.0.0.1",
06567          port=port,
06568          log_level="warning",
06569          access_log=False,
06570      )
06571      return uvicorn.Server(config), server_socket, port
06572  def wait_for_server(url: str, timeout_seconds: float = 15) -> None:
06573      deadline = time.monotonic() + timeout_seconds
06574      while time.monotonic() < deadline:
06575          try:
06576              with urlopen(f"{url}/health", timeout=1) as response:
06577                  if response.status == 200:

```

```

06578         return
06579     except (OSError, URLError):
06580         time.sleep(0.1)
06581     raise RuntimeError("本地分析服务启动超时。")
06582 def run_smoke_test(url: str) -> None:
06583     with urlopen(f"{url}/health", timeout=5) as response:
06584         if response.status != 200:
06585             raise RuntimeError("健康检查失败。")
06586     with urlopen(url, timeout=5) as response:
06587         page = response.read(4096)
06588         if response.status != 200 or b"<html" not in page.lower():
06589             raise RuntimeError("Vue 首页没有正确打包。")
06590 def main() -> int:
06591     configure_logging()
06592     instance_handle = acquire_single_instance()
06593     if instance_handle is None:
06594         show_error("DataEX-G 已经在运行。")
06595         return 1
06596     if not (frontend_directory() / "index.html").is_file():
06597         show_error("没有找到前端页面，请先运行前端生产构建。")
06598     release_single_instance(instance_handle)
06599     return 1
06600     server, server_socket, port = create_server()
06601     server_thread = threading.Thread(
06602         target=server.run,
06603         kwargs={"sockets": [server_socket]},
06604         name="local-api",
06605         daemon=True,
06606     )
06607     server_thread.start()
06608     url = f"http://127.0.0.1:{port}"
06609     try:
06610         wait_for_server(url)
06611         if os.environ.get("DATA_ANALYSIS_DESKTOP_SMOKE_TEST") == "1":
06612             run_smoke_test(url)
06613         return 0
06614     webview.create_window(
06615         APP_TITLE,
06616         url,
06617         width=1280,
06618         height=820,
06619         min_size=(960, 640),
06620         background_color="#edf2ed",
06621     )
06622     webview.start(
06623         debug=False,
06624         private_mode=False,
06625         storage_path=str(local_data_directory() / "webview"),
06626     )
06627     return 0
06628 except (OSError, RuntimeError) as exc:
06629     show_error(f"程序启动失败: {exc}")
06630     return 1
06631 finally:

```

```
06632     server.should_exit = True
06633     server_thread.join(timeout=5)
06634     server_socket.close()
06635     release_single_instance(instance_handle)
06636 if __name__ == "__main__":
06637     raise SystemExit(main())
06638 # Source file: backend/src/backend/resources.py
06639 import sys
06640 from pathlib import Path
06641 def application_root() -> Path:
06642     bundled_root = getattr(sys, "_MEIPASS", None)
06643     if bundled_root:
06644         return Path(bundled_root)
06645     return Path(__file__).resolve().parents[3]
06646 def frontend_directory() -> Path:
06647     root = application_root()
06648     if getattr(sys, "frozen", False):
06649         return root / "frontend"
06650     return root / "frontend" / "dist"
```